

✓ UNSW-NB15 Network Intrusion Detection using Autoencoders

CMPE257 Machine Learning Project

This notebook implements anomaly detection for network intrusion detection using autoencoder neural networks on the UNSW-NB15 dataset.

Table of Contents

1. [Setup and Installation](#)
2. [Data Loading from Google Drive](#)
3. [Data Preprocessing](#)
4. [Autoencoder Model](#)
5. [Model Architecture and Training Configuration](#)
 - 5.1-5.6: Architecture, Loss Function, Training Config
 - 5.7: **Complete Pipeline Walkthrough** (Step-by-Step Example)
6. [Training](#)
7. [Evaluation and Testing](#)
8. [Inference Pipeline](#)

✓ 1. Setup and Installation {#setup}

```
# Install required packages (uncomment if needed)
# !pip install tensorflow pandas numpy scikit-learn matplotlib seaborn imbalanced-learn
```

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import os
import json
import time
from datetime import datetime

# Machine Learning libraries
from sklearn.preprocessing import StandardScaler, OneHotEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import (
    classification_report, confusion_matrix, roc_auc_score,
    roc_curve, auc, precision_recall_curve, f1_score,
    accuracy_score, precision_score, recall_score, silhouette_score
)

# Data Mining: Dimensionality Reduction
from sklearn.decomposition import PCA
from sklearn.feature_selection import mutual_info_classif

# Data Mining: Outlier Detection Methods
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.svm import OneClassSVM
from sklearn.neighbors import LocalOutlierFactor

# Data Mining: Clustering
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.mixture import GaussianMixture

# Data Mining: Data Balancing (SMOTE)
try:
    from imblearn.over_sampling import SMOTE, ADASYN
    from imblearn.under_sampling import RandomUnderSampler
    IMBLEARN_AVAILABLE = True
except ImportError:
    print("Warning: imbalanced-learn not installed. SMOTE/ADASYN will be skipped.")
```

```

print("Install with: pip install imbalanced-learn")
IMBLEARN_AVAILABLE = False

# TensorFlow and Keras
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping, ReduceLRonPlateau, ModelCheckpoint

# Configuration
warnings.filterwarnings('ignore')
sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

# Set random seeds for reproducibility
RANDOM_SEED = 42
np.random.seed(RANDOM_SEED)
tf.random.set_seed(RANDOM_SEED)

print(f"TensorFlow version: {tf.__version__}")
print(f"NumPy version: {np.__version__}")
print(f"Pandas version: {pd.__version__}")

TensorFlow version: 2.19.0
NumPy version: 2.0.2
Pandas version: 2.2.2

```

2. Mount Google Drive and Load Data {#data-loading}

Upload your UNSW-NB15 CSV files to Google Drive first:

- UNSW-NB15_1.csv
- UNSW-NB15_2.csv
- UNSW-NB15_3.csv
- UNSW-NB15_4.csv
- NUSW-NB15_features.csv

```

# Mount Google Drive (for Colab)
try:
    from google.colab import drive
    drive.mount('/content/drive')

    # IMPORTANT: Update this path to match your Google Drive folder
    DATA_DIR = '/content/drive/MyDrive/UNSW-NB15-Data/'
    FEATURE_FILE = '/content/drive/MyDrive/UNSW-NB15-Data/NUSW-NB15_features.csv'
    IS_COLAB = True

    print(f"✓ Google Drive mounted successfully")
    print(f"Data directory: {DATA_DIR}")
except:
    print("Not running in Google Colab. Using local paths.")
    DATA_DIR = 'data/'
    FEATURE_FILE = 'NUSW-NB15_features.csv'
    IS_COLAB = False

```

```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True)
✓ Google Drive mounted successfully
Data directory: /content/drive/MyDrive/UNSW-NB15-Data/

```

3. Data Preprocessing Pipeline {#preprocessing}

This class handles loading, cleaning, and preprocessing the UNSW-NB15 dataset.

```

class UNSWDataPreprocessor:
    """Data preprocessor for UNSW-NB15 dataset."""

    def __init__(self, onehot_threshold=15):
        """
        Args:
            onehot_threshold: Max unique values for one-hot encoding.
        """

```

```

        Features with more unique values use frequency encoding.
    """
    self.scaler = StandardScaler() # Only for numerical features
    self.onehot_encoder = OneHotEncoder(sparse_output=False, handle_unknown='ignore')
    self.onehot_threshold = onehot_threshold
    self.feature_names = None
    self.numerical_features = None
    self.categorical_features = None
    self.low_cardinality_features = [] # For one-hot encoding
    self.high_cardinality_features = [] # For frequency encoding
    self.frequency_maps = {} # Store frequency mappings
    self.encoded_feature_names = None # Track final feature names after encoding

def load_data(self, data_dir=DATA_DIR, sample_size=None):
    """Load all UNSW-NB15 CSV files and combine them."""
    print("Loading UNSW-NB15 dataset...")

    # Load feature names
    try:
        features_df = pd.read_csv(FEATURE_FILE, encoding='latin-1')
        self.feature_names = features_df['Name'].tolist()
        print(f"Loaded {len(self.feature_names)} feature names")
    except:
        print("Warning: Could not load feature names, using generic names")
        self.feature_names = [f'feature_{i}' for i in range(49)]

    # Load all data files
    data_files = [
        f'{data_dir}UNSW-NB15_1.csv',
        f'{data_dir}UNSW-NB15_2.csv',
        f'{data_dir}UNSW-NB15_3.csv',
        f'{data_dir}UNSW-NB15_4.csv'
    ]

    dfs = []
    for file_path in data_files:
        try:
            df = pd.read_csv(file_path, header=None, low_memory=False)
            df.columns = self.feature_names
            dfs.append(df)
            print(f"Loaded {os.path.basename(file_path)}: {df.shape}")
        except Exception as e:
            print(f"Error loading {file_path}: {e}")

    # Combine all dataframes
    combined_df = pd.concat(dfs, ignore_index=True)
    print(f"Combined dataset shape: {combined_df.shape}")

    # Sample data if requested
    if sample_size and sample_size < len(combined_df):
        combined_df = combined_df.sample(n=sample_size, random_state=RANDOM_SEED)
        print(f"Sampled dataset shape: {combined_df.shape}")

    return combined_df

def identify_feature_types(self, df):
    """Identify numerical and categorical features."""
    self.numerical_features = []
    self.categorical_features = []

    for col in df.columns:
        if col in ['Label', 'attack_cat']:
            continue
        elif df[col].dtype in ['int64', 'float64']:
            self.numerical_features.append(col)
        else:
            self.categorical_features.append(col)

    print(f"Numerical features: {len(self.numerical_features)}")
    print(f"Categorical features: {len(self.categorical_features)}")

def clean_data(self, df, clip_outliers=True):
    """Clean the dataset by handling missing values and outliers.

    Data Mining Techniques Applied:
    1. Missing value imputation (median for numerical, mode for categorical)
    2. Infinite value handling
    """

```

```

3. Outlier clipping using IQR method (prevents extreme values from distorting analysis)
"""
print("Cleaning data...")

# Handle missing values
missing_counts = df.isnull().sum()
if missing_counts.sum() > 0:
    print(f" Missing values found: {missing_counts.sum()}")

    for col in df.columns:
        if df[col].dtype in ['int64', 'float64']:
            df[col].fillna(df[col].median(), inplace=True)
        else:
            df[col].fillna(df[col].mode()[0] if not df[col].mode().empty else 'unknown', inplace=True)

# Handle infinite values
df.replace([np.inf, -np.inf], np.nan, inplace=True)
df.fillna(df.median(numeric_only=True), inplace=True)

# Clip outliers using IQR method for numerical features
if clip_outliers:
    numerical_cols = df.select_dtypes(include=['int64', 'float64']).columns
    numerical_cols = [c for c in numerical_cols if c not in ['Label', 'attack_cat']]

    outliers_clipped = 0
    for col in numerical_cols:
        Q1 = df[col].quantile(0.01) # 1st percentile
        Q3 = df[col].quantile(0.99) # 99th percentile

        # Count outliers before clipping
        outliers_clipped += ((df[col] < Q1) | (df[col] > Q3)).sum()

        # Clip values to [Q1, Q3] range
        df[col] = df[col].clip(lower=Q1, upper=Q3)

    print(f" Outliers clipped (1st-99th percentile): {outliers_clipped} values")

return df

def encode_categorical_features(self, df, fit_encoder=True):
    """Encode categorical features using hybrid approach:

    - One-hot encoding for LOW cardinality features (< threshold unique values)
      → No ordinal bias, each category equidistant
    - Frequency encoding for HIGH cardinality features (>= threshold unique values)
      → Prevents dimensionality explosion, captures category importance

    This is a key DATA MINING decision: choosing the right encoding based on data characteristics.
    """
    print("Encoding categorical features (Hybrid Approach)...")

    df_encoded = df.copy()

    # Fill missing categorical values
    for col in self.categorical_features:
        if col in df_encoded.columns:
            df_encoded[col] = df_encoded[col].fillna('unknown').astype(str)

    # Separate features by cardinality
    if fit_encoder:
        self.low_cardinality_features = []
        self.high_cardinality_features = []

        for col in self.categorical_features:
            n_unique = df_encoded[col].nunique()
            if n_unique <= self.onehot_threshold:
                self.low_cardinality_features.append(col)
            else:
                self.high_cardinality_features.append(col)

        print(f" Low cardinality (one-hot, <= {self.onehot_threshold} unique): {self.low_cardinality_features}")
        print(f" High cardinality (frequency): {self.high_cardinality_features}")

    encoded_parts = []
    self.onehot_feature_names = []

    # 1. One-hot encode low cardinality features

```

```

if self.low_cardinality_features:
    cat_data_low = df_encoded[self.low_cardinality_features]
    if fit_encoder:
        onehot_encoded = self.onehot_encoder.fit_transform(cat_data_low)
        self.onehot_feature_names = self.onehot_encoder.get_feature_names_out(self.low_cardinality_features)
    else:
        onehot_encoded = self.onehot_encoder.transform(cat_data_low)
    encoded_parts.append(onehot_encoded)
    print(f" One-hot encoded: {len(self.low_cardinality_features)} features → {onehot_encoded.shape[1]} col

# 2. Frequency encode high cardinality features
self.freq_feature_names = []
if self.high_cardinality_features:
    freq_encoded_list = []
    for col in self.high_cardinality_features:
        if fit_encoder:
            # Calculate frequency (proportion) of each category
            freq_map = df_encoded[col].value_counts(normalize=True).to_dict()
            self.frequency_maps[col] = freq_map
        else:
            freq_map = self.frequency_maps.get(col, {})

        # Map categories to their frequencies (unknown categories get 0)
        freq_values = df_encoded[col].map(freq_map).fillna(0).values.reshape(-1, 1)
        freq_encoded_list.append(freq_values)
        self.freq_feature_names.append(f"{col}_freq")

    freq_encoded = np.hstack(freq_encoded_list)
    encoded_parts.append(freq_encoded)
    print(f" Frequency encoded: {len(self.high_cardinality_features)} features → {freq_encoded.shape[1]} co

# Combine all encoded features
if encoded_parts:
    cat_encoded = np.hstack(encoded_parts)
else:
    cat_encoded = np.array([]).reshape(len(df), 0)

print(f" Total categorical features after encoding: {cat_encoded.shape[1]}")

return cat_encoded

def preprocess_features(self, df, fit_scaler=True):
    """Preprocess features for autoencoder training.

    Data Mining Pipeline:
    1. Clean data (handle missing values, infinite values)
    2. Identify feature types (numerical vs categorical)
    3. Encode categorical features:
        - One-hot for low cardinality (no scaling needed – binary)
        - Frequency encoding for high cardinality (captures importance)
    4. Standardize numerical features only (z-score normalization)
    5. Concatenate: [scaled_numerical | encoded_categorical]
    """
    print("Preprocessing features...")

    # Clean data
    df_clean = self.clean_data(df.copy())

    # Identify feature types
    self.identify_feature_types(df_clean)

    # Extract numerical features (exclude target columns)
    num_cols = [col for col in self.numerical_features if col not in ['Label', 'attack_cat']]
    X_numerical = df_clean[num_cols].values

    # Scale numerical features only
    if fit_scaler:
        X_num_scaled = self.scaler.fit_transform(X_numerical)
    else:
        X_num_scaled = self.scaler.transform(X_numerical)

    # Encode categorical features (hybrid: one-hot + frequency)
    X_categorical = self.encode_categorical_features(df_clean, fit_encoder=fit_scaler)

    # Concatenate: [numerical (scaled) | categorical (encoded)]
    X_combined = np.hstack([X_num_scaled, X_categorical])

```

```

# Track feature names for interpretability
self.encoded_feature_names = num_cols + self.onehot_feature_names + self.freq_feature_names

print(f" Numerical features (scaled): {X_num_scaled.shape[1]}")
print(f" Categorical features (encoded): {X_categorical.shape[1]}")
print(f" Total features: {X_combined.shape[1]}")

return X_combined

def prepare_autoencoder_data(self, data_dir=DATA_DIR, sample_size=None, test_size=0.2):
    """Complete data preparation pipeline for autoencoder training."""
    # Load data
    df = self.load_data(data_dir, sample_size)

    # Separate normal and attack data
    normal_df = df[df['Label'] == 0].copy()
    attack_df = df[df['Label'] == 1].copy()

    print(f"\nNormal samples: {len(normal_df)}")
    print(f"Attack samples: {len(attack_df)}")

    # Split normal data for training and validation
    normal_train, normal_test = train_test_split(
        normal_df, test_size=test_size, random_state=RANDOM_SEED
    )

    # Combine normal test data with all attack data for testing
    test_df = pd.concat([normal_test, attack_df], ignore_index=True)

    # Preprocess features
    X_train_normal = self.preprocess_features(normal_train, fit_scaler=True)
    X_test = self.preprocess_features(test_df, fit_scaler=False)

    y_test = test_df['Label'].values
    attack_cats_test = test_df['attack_cat'].values

    print(f"\nTraining set (NORMAL ONLY): {X_train_normal.shape[0]} samples")
    print(f"Test set (Normal + Attack): {X_test.shape[0]} samples")
    print(f"Normal samples in test: {np.sum(y_test == 0)}")
    print(f"Attack samples in test: {np.sum(y_test == 1)}")

    return X_train_normal, X_test, y_test, attack_cats_test, test_df

print("✓ Data preprocessing pipeline defined")

```

✓ Data preprocessing pipeline defined

4. Autoencoder Model {#model}

We implement an autoencoder for anomaly detection with a simple encoder-decoder architecture using dropout regularization.

```

class Autoencoder:
    """Autoencoder for anomaly detection."""

    def __init__(self, input_dim, encoding_dim=32, hidden_dims=[64, 32]):
        self.input_dim = input_dim
        self.encoding_dim = encoding_dim
        self.hidden_dims = hidden_dims
        self.autoencoder = None
        self.encoder = None
        self.threshold = None
        self.history = None

    def build_autoencoder(self):
        """Build the autoencoder architecture."""
        input_layer = keras.Input(shape=(self.input_dim,))

        # Encoder
        encoded = input_layer
        for dim in self.hidden_dims:
            encoded = layers.Dense(dim, activation='relu')(encoded)
            encoded = layers.Dropout(0.2)(encoded)

        encoded = layers.Dense(self.encoding_dim, activation='relu', name='encoded')(encoded)

```

```

# Decoder
decoded = encoded
for dim in reversed(self.hidden_dims):
    decoded = layers.Dense(dim, activation='relu')(decoded)
    decoded = layers.Dropout(0.2)(decoded)

decoded = layers.Dense(self.input_dim, activation='sigmoid')(decoded)

self.autoencoder = keras.Model(input_layer, decoded)
self.encoder = keras.Model(input_layer, encoded)

def compile_model(self, optimizer='adam', loss='mse'):
    """Compile the autoencoder model."""
    if self.autoencoder is None:
        self.build_autoencoder()

    self.autoencoder.compile(optimizer=optimizer, loss=loss, metrics=['mae'])
    print("✓ Autoencoder compiled successfully!")
    return self.autoencoder

def train(self, X_train, X_val=None, epochs=100, batch_size=32, verbose=1):
    """Train the autoencoder."""
    if self.autoencoder is None:
        self.compile_model()

    validation_split = None if X_val is not None else 0.2

    self.history = self.autoencoder.fit(
        X_train, X_train,
        epochs=epochs,
        batch_size=batch_size,
        validation_data=(X_val, X_val) if X_val is not None else None,
        validation_split=validation_split,
        verbose=verbose,
        shuffle=True
    )

    print("✓ Training completed!")
    return self.history

def predict_anomaly_scores(self, X):
    """Calculate reconstruction error as anomaly scores."""
    if self.autoencoder is None:
        raise ValueError("Model must be trained first!")

    X_pred = self.autoencoder.predict(X, verbose=0)
    mse = np.mean(np.power(X - X_pred, 2), axis=1)

    return mse

def set_threshold(self, X_normal, percentile=95):
    """Set anomaly threshold based on normal data."""
    normal_scores = self.predict_anomaly_scores(X_normal)
    self.threshold = np.percentile(normal_scores, percentile)
    print(f"Anomaly threshold set to: {self.threshold:.6f}")
    return self.threshold

def predict_anomalies(self, X):
    """Predict anomalies based on reconstruction error threshold."""
    if self.threshold is None:
        raise ValueError("Threshold must be set first!")

    scores = self.predict_anomaly_scores(X)
    predictions = (scores > self.threshold).astype(int)

    return predictions, scores

def evaluate(self, X_test, y_test):
    """Evaluate the model performance."""
    predictions, scores = self.predict_anomalies(X_test)

    print("\n" + "="*60)
    print("CLASSIFICATION REPORT")
    print("="*60)
    print(classification_report(y_test, predictions, target_names=['Normal', 'Anomaly']))

```

```

cm = confusion_matrix(y_test, predictions)
print("\nConfusion Matrix:")
print(cm)

roc_auc = roc_auc_score(y_test, scores)
accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)

print(f"\nROC AUC: {roc_auc:.4f}")
print(f"Accuracy: {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall: {recall:.4f}")
print(f"F1-Score: {f1:.4f}")

return {
    'predictions': predictions,
    'scores': scores,
    'roc_auc': roc_auc,
    'accuracy': accuracy,
    'precision': precision,
    'recall': recall,
    'f1_score': f1,
    'confusion_matrix': cm
}

print("✓ Autoencoder model defined")

```

✓ Autoencoder model defined

5. Load and Prepare Data

Now let's load the data and prepare it for training. Adjust `SAMPLE_SIZE` based on your needs:

- Use `50000` for quick testing
- Use `None` for full dataset (may take longer)

Double-click (or enter) to edit

```

# Configuration
# =====
# SAMPLE SIZE CONFIGURATION
# =====
# Set the total sample size - this controls how much data we work with
# The data will be split 80% train, 20% test
SAMPLE_SIZE = 50000 # Total records to use (None for full dataset)
EPOCHS = 50
BATCH_SIZE = 128

# Initialize preprocessor and load data
preprocessor = UNSWDataPreprocessor()
X_train_normal, X_test, y_test, attack_cats_test, test_df = preprocessor.prepare_autoencoder_data(
    data_dir=DATA_DIR,
    sample_size=SAMPLE_SIZE,
    test_size=0.2
)

# Split training data for validation (for autoencoder early stopping)
X_train_full, X_val = train_test_split(X_train_normal, test_size=0.2, random_state=RANDOM_SEED)

print(f"\n{'='*60}")
print("DATA READY FOR TRAINING")
print(f"{'='*60}")
print(f"Training set: {X_train_full.shape}")
print(f"Validation set: {X_val.shape}")
print(f"Test set: {X_test.shape}")
print(f"Input features: {X_train_full.shape[1]}")

# =====
# CONSISTENT DATA FOR ALL MODELS
# =====

```

```
# Use the same data across all models for fair comparison
X_train_sample = X_train_full # Use all training data
X_test_sample = X_test        # Use all test data
y_test_sample = y_test

print(f"\n🌈 Data for all models:")
print(f"  Training samples: {len(X_train_sample)}")
print(f"  Test samples: {len(X_test_sample)}")
print(f"  Test normal: {np.sum(y_test_sample == 0)}, Test attacks: {np.sum(y_test_sample == 1)}")
```

```
Loading UNSW-NB15 dataset...
Loaded 49 feature names
Loaded UNSW-NB15_1.csv: (700001, 49)
Loaded UNSW-NB15_2.csv: (700001, 49)
Loaded UNSW-NB15_3.csv: (700001, 49)
Loaded UNSW-NB15_4.csv: (440044, 49)
Combined dataset shape: (2540047, 49)
Sampled dataset shape: (50000, 49)

Normal samples: 43720
Attack samples: 6280
Preprocessing features...
Cleaning data...
  Missing values found: 69408
  Outliers clipped (1st-99th percentile): 12203 values
Numerical features: 39
Categorical features: 8
Encoding categorical features (Hybrid Approach)...
  Low cardinality (one-hot, <= 15 unique): ['proto', 'state', 'service', 'ct_ftp_cmd']
  High cardinality (frequency): ['srcip', 'sport', 'dstip', 'dport']
  One-hot encoded: 4 features → 32 columns
  Frequency encoded: 4 features → 4 columns
  Total categorical features after encoding: 36
  Numerical features (scaled): 39
  Categorical features (encoded): 36
  Total features: 75
Preprocessing features...
Cleaning data...
  Missing values found: 28754
  Outliers clipped (1st-99th percentile): 4963 values
Numerical features: 39
Categorical features: 8
Encoding categorical features (Hybrid Approach)...
  One-hot encoded: 4 features → 32 columns
  Frequency encoded: 4 features → 4 columns
  Total categorical features after encoding: 36
  Numerical features (scaled): 39
  Categorical features (encoded): 36
  Total features: 75

Training set (NORMAL ONLY): 34976 samples
Test set (Normal + Attack): 15024 samples
Normal samples in test: 8744
Attack samples in test: 6280

=====
DATA READY FOR TRAINING
=====
Training set: (27980, 75)
Validation set: (6996, 75)
Test set: (15024, 75)
Input features: 75

🌈 Data for all models:
  Training samples: 27980
  Test samples: 15024
  Test normal: 8744, Test attacks: 6280
```

✓ Part 2: Advanced Data Mining Techniques

This section demonstrates various **data mining techniques** applied to network intrusion detection:

1. **Exploratory Data Analysis** - Correlation analysis, feature distributions
2. **Dimensionality Reduction** - PCA visualization
3. **Feature Importance** - Random Forest, Mutual Information
4. **Alternative Anomaly Detection** - Isolation Forest, One-Class SVM, LOF
5. **Clustering Analysis** - K-Means, DBSCAN, GMM

6. Exploratory Data Analysis (EDA)

6.1 Feature Correlation Analysis

Correlation analysis helps identify:

- **Redundant features** (highly correlated) that can be removed
- **Feature relationships** that reveal data patterns
- **Multicollinearity** issues for certain algorithms

```
# Correlation Analysis on Training Data
print("=" * 60)
print("CORRELATION ANALYSIS")
print("=" * 60)

# Create DataFrame from training data with feature names
if preprocessor.encoded_feature_names is not None:
    feature_names = preprocessor.encoded_feature_names
else:
    feature_names = [f'feature_{i}' for i in range(X_train.shape[1])]

# For correlation, use only numerical features (first part before encoded categorical)
num_feature_names = [f for f in preprocessor.numerical_features if f not in ['Label', 'attack_cat']]
num_features = len(num_feature_names)
X_train_numerical = X_train[:, :num_features]

print(f"Analyzing {num_features} numerical features...")

# Calculate correlation matrix
correlation_matrix = np.corrcoef(X_train_numerical.T)

# Handle edge case where correlation matrix might be 1D (single feature)
if correlation_matrix.ndim == 0:
    correlation_matrix = np.array([[correlation_matrix]])

# Plot correlation heatmap (show top 15 features for print-friendly size)
n_show = min(15, num_features)
fig, ax = plt.subplots(figsize=(10, 8)) # Smaller, print-friendly size
corr_subset = correlation_matrix[:n_show, :n_show]
mask = np.triu(np.ones_like(corr_subset, dtype=bool), k=1)
sns.heatmap(corr_subset, mask=mask, cmap='coolwarm', center=0,
            annot=True, fmt='.2f', square=True, linewidths=0.5,
            annot_kws={'size': 7}, # Smaller annotation text
            xticklabels=num_feature_names[:n_show], yticklabels=num_feature_names[:n_show],
            cbar_kws={'shrink': 0.7}, ax=ax)
ax.set_title(f'Feature Correlation Heatmap (Top {n_show} Numerical Features)', fontsize=12, fontweight='bold')
ax.set_xticklabels(ax.get_xticklabels(), rotation=45, ha='right', fontsize=8)
ax.set_yticklabels(ax.get_yticklabels(), rotation=0, fontsize=8)
# Add extra padding to prevent cutoff when printing
plt.subplots_adjust(bottom=0.2, left=0.15)
plt.tight_layout()
plt.show()

# Find highly correlated feature pairs
print("\n🌈 Highly Correlated Feature Pairs (|r| > 0.8):")
print("-" * 50)
high_corr_pairs = []
n_corr = correlation_matrix.shape[0]
for i in range(n_corr):
    for j in range(i+1, n_corr):
        if abs(correlation_matrix[i, j]) > 0.8:
            high_corr_pairs.append((num_feature_names[i], num_feature_names[j], correlation_matrix[i, j]))

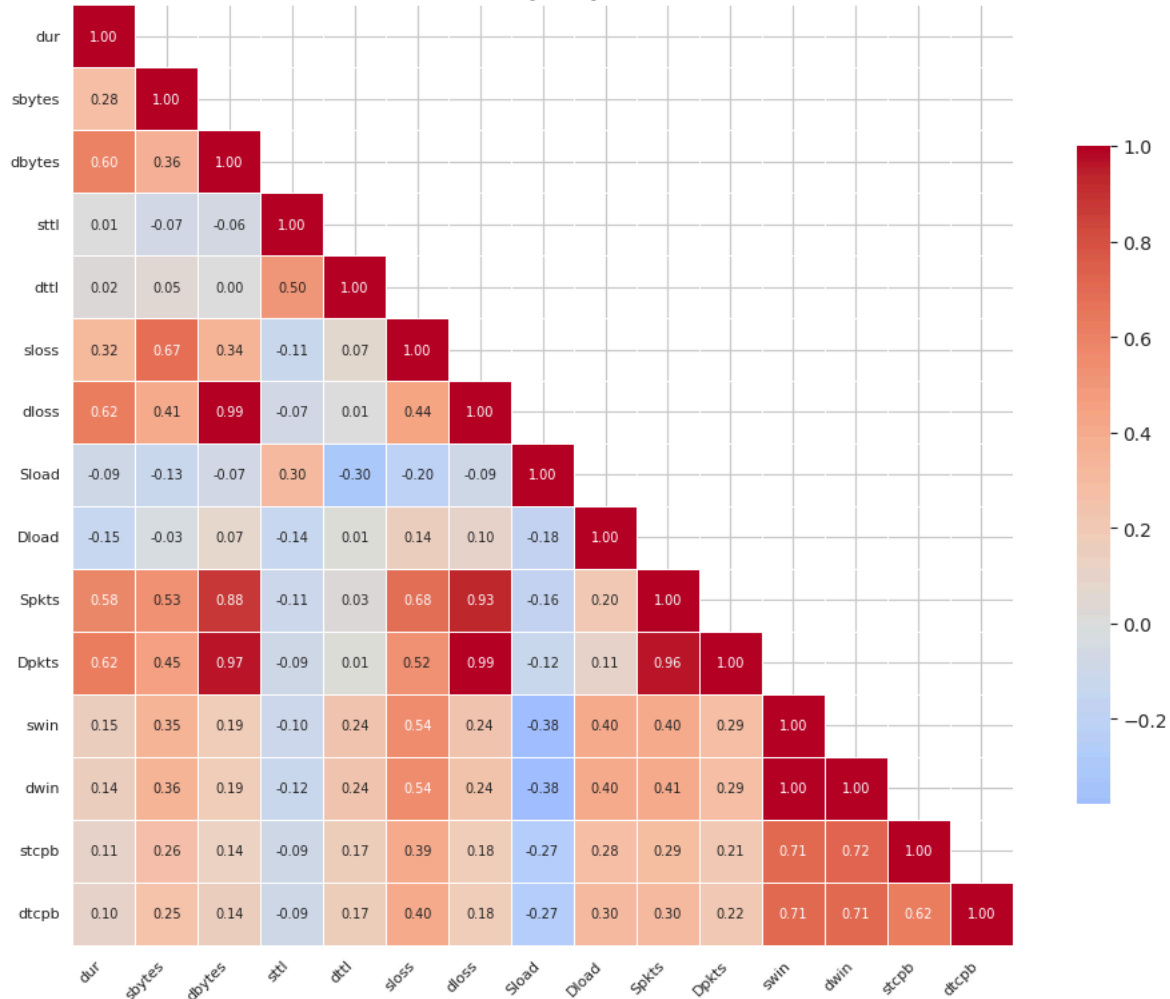
high_corr_pairs.sort(key=lambda x: abs(x[2]), reverse=True)
for feat1, feat2, corr in high_corr_pairs[:10]:
    print(f" {feat1} <-> {feat2}: {corr:.3f}")

print(f"\n✓ Found {len(high_corr_pairs)} highly correlated pairs (potential redundancy)")
```

CORRELATION ANALYSIS

Analyzing 39 numerical features...

Feature Correlation Heatmap (Top 15 Numerical Features)



Highly Correlated Feature Pairs ($|r| > 0.8$):

```
trans_depth <=> ct_flw_http_mthd: 1.000
Stime <=> Ltime: 1.000
swin <=> dwin: 0.997
tcprrt <=> synack: 0.994
dloss <=> Dpkts: 0.992
dbytes <=> dloss: 0.990
tcprrt <=> ackdat: 0.989
synack <=> ackdat: 0.973
dbytes <=> Dpkts: 0.965
Spkts <=> Dpkts: 0.965
```

7. Dimensionality Reduction with PCA

Principal Component Analysis (PCA) is a linear dimensionality reduction technique that:

- Projects data onto orthogonal axes of maximum variance
- Helps visualize high-dimensional data in 2D/3D
- Can reveal cluster structures and separability between classes

```
# PCA for Visualization and Analysis
print("=" * 60)
print("PCA - DIMENSIONALITY REDUCTION")
print("=" * 60)

# Apply PCA to test data (which has both normal and attack samples)
pca_full = PCA()
X_test_pca_full = pca_full.fit_transform(X_test)
```

```

# Explained variance ratio
explained_variance = pca_full.explained_variance_ratio_
cumulative_variance = np.cumsum(explained_variance)

# Plot explained variance
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Plot 1: Explained variance per component
ax1 = axes[0]
ax1.bar(range(1, min(31, len(explained_variance)+1)), explained_variance[:30], alpha=0.7, color='steelblue')
ax1.set_xlabel('Principal Component')
ax1.set_ylabel('Explained Variance Ratio')
ax1.set_title('Variance Explained by Each Component', fontweight='bold')
ax1.grid(True, alpha=0.3)

# Plot 2: Cumulative explained variance
ax2 = axes[1]
ax2.plot(range(1, len(cumulative_variance)+1), cumulative_variance, 'b-o', markersize=3)
ax2.axhline(y=0.95, color='r', linestyle='--', label='95% Variance')
ax2.axhline(y=0.99, color='g', linestyle='--', label='99% Variance')
n_95 = np.argmax(cumulative_variance >= 0.95) + 1
n_99 = np.argmax(cumulative_variance >= 0.99) + 1
ax2.axvline(x=n_95, color='r', linestyle=':', alpha=0.5)
ax2.axvline(x=n_99, color='g', linestyle=':', alpha=0.5)
ax2.set_xlabel('Number of Components')
ax2.set_ylabel('Cumulative Explained Variance')
ax2.set_title('Cumulative Variance Explained', fontweight='bold')
ax2.legend()
ax2.grid(True, alpha=0.3)

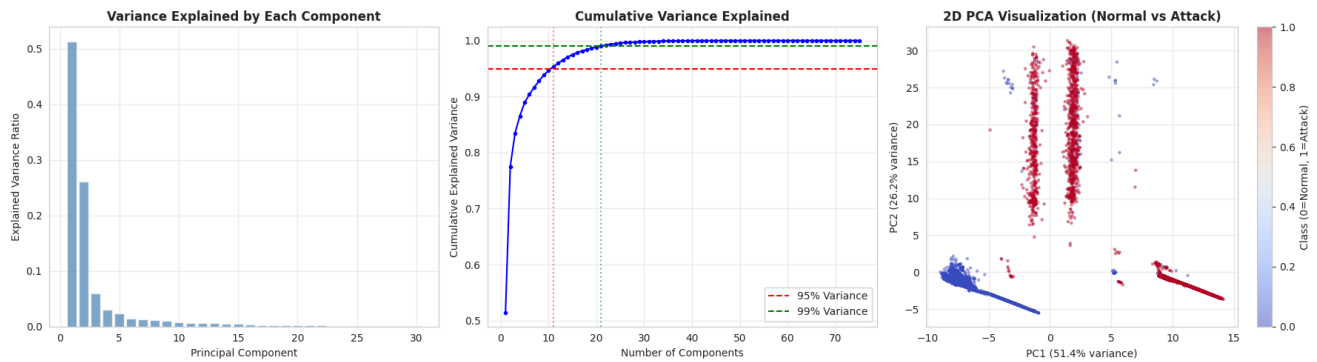
# Plot 3: 2D PCA visualization
ax3 = axes[2]
pca_2d = PCA(n_components=2)
X_test_2d = pca_2d.fit_transform(X_test)
scatter = ax3.scatter(X_test_2d[:, 0], X_test_2d[:, 1], c=y_test, cmap='coolwarm',
                      alpha=0.5, s=10, edgecolors='none')
ax3.set_xlabel(f'PC1 ({pca_2d.explained_variance_ratio_[0]*100:.1f}% variance)')
ax3.set_ylabel(f'PC2 ({pca_2d.explained_variance_ratio_[1]*100:.1f}% variance)')
ax3.set_title('2D PCA Visualization (Normal vs Attack)', fontweight='bold')
plt.colorbar(scatter, ax=ax3, label='Class (0=Normal, 1=Attack)')
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Print summary
print(f"\n🇩🇪 PCA Analysis Summary:")
print(f" - Total features: {X_test.shape[1]}")
print(f" - Components for 95% variance: {n_95}")
print(f" - Components for 99% variance: {n_99}")
print(f" - First 2 components explain: {cumulative_variance[1]*100:.1f}% variance")
print(f"\n🇩🇪 PCA reveals potential dimensionality reduction from {X_test.shape[1]} to {n_95} features")

```

PCA – DIMENSIONALITY REDUCTION



PCA Analysis Summary:

- Total features: 75
- Components for 95% variance: 11
- Components for 99% variance: 21
- First 2 components explain: 77.5% variance

✓ PCA reveals potential dimensionality reduction from 75 to 11 features

8. Feature Importance Analysis

Understanding which features contribute most to anomaly detection:

8.1 Random Forest Feature Importance

Uses a supervised approach to rank features by their contribution to classification.

8.2 Mutual Information

Measures the dependency between each feature and the target variable.

```
# Feature Importance Analysis
print("=" * 60)
print("FEATURE IMPORTANCE ANALYSIS")
print("=" * 60)

# Use consistent samples defined earlier
print(f"Using test samples: {len(X_test_sample)}")

# 1. Random Forest Feature Importance
print("\n🌲 Training Random Forest for Feature Importance...")
rf = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=RANDOM_SEED, n_jobs=-1)
rf.fit(X_test_sample, y_test_sample)
rf_importance = rf.feature_importances_

# 2. Mutual Information
print("📊 Calculating Mutual Information...")
mi_scores = mutual_info_classif(X_test_sample, y_test_sample, random_state=RANDOM_SEED)

# Create feature importance DataFrame
if len(feature_names) == X_test.shape[1]:
    feat_names = feature_names
else:
    feat_names = [f'feature_{i}' for i in range(X_test.shape[1])]

importance_df = pd.DataFrame({
```

```

        'Feature': feat_names,
        'RF_Importance': rf_importance,
        'Mutual_Info': mi_scores
    })

# Sort by RF importance
importance_df = importance_df.sort_values('RF_Importance', ascending=False)

# Plot top 20 features
fig, axes = plt.subplots(1, 2, figsize=(16, 8))

# Random Forest Importance
ax1 = axes[0]
top_rf = importance_df.head(20)
colors = plt.cm.Blues(np.linspace(0.4, 0.9, 20))
ax1.barh(range(20), top_rf['RF_Importance'].values[::-1], color=colors[::-1])
ax1.set_yticks(range(20))
ax1.set_yticklabels(top_rf['Feature'].values[::-1], fontsize=9)
ax1.set_xlabel('Importance Score')
ax1.set_title('Top 20 Features - Random Forest Importance', fontweight='bold')
ax1.grid(True, alpha=0.3, axis='x')

# Mutual Information
ax2 = axes[1]
mi_sorted = importance_df.sort_values('Mutual_Info', ascending=False).head(20)
colors = plt.cm.Greens(np.linspace(0.4, 0.9, 20))
ax2.barh(range(20), mi_sorted['Mutual_Info'].values[::-1], color=colors[::-1])
ax2.set_yticks(range(20))
ax2.set_yticklabels(mi_sorted['Feature'].values[::-1], fontsize=9)
ax2.set_xlabel('Mutual Information Score')
ax2.set_title('Top 20 Features - Mutual Information', fontweight='bold')
ax2.grid(True, alpha=0.3, axis='x')

plt.tight_layout()
plt.show()

# Print top features
print("\n🇩🇪 Top 10 Features by Random Forest Importance:")
for i, row in importance_df.head(10).iterrows():
    print(f" {row['Feature']}: {row['RF_Importance']:.4f}")

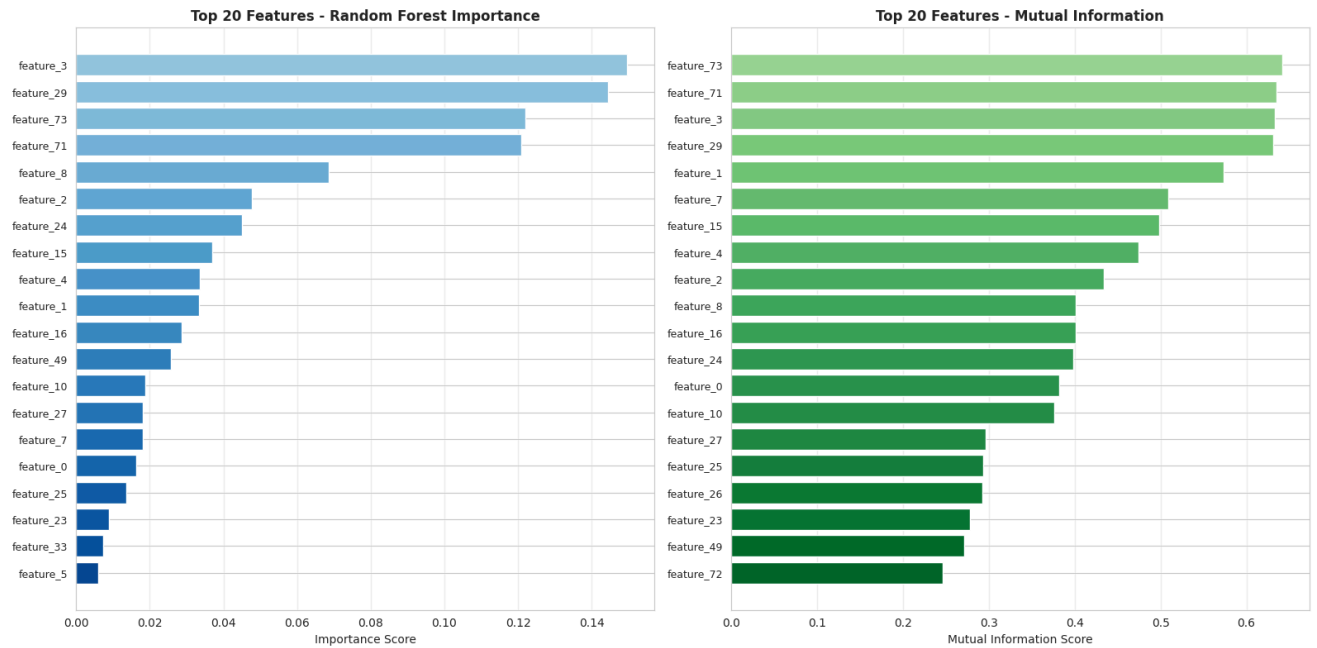
print("\n🇩🇪 Top 10 Features by Mutual Information:")
mi_top = importance_df.sort_values('Mutual_Info', ascending=False).head(10)
for i, row in mi_top.iterrows():
    print(f" {row['Feature']}: {row['Mutual_Info']:.4f}")

```

FEATURE IMPORTANCE ANALYSIS

Using test samples: 15024

🌲 Training Random Forest for Feature Importance...
🇩🇪 Calculating Mutual Information...



🇩🇪 Top 10 Features by Random Forest Importance:

feature_3: 0.1496
feature_29: 0.1445
feature_73: 0.1220
feature_71: 0.1209
feature_8: 0.0686
feature_2: 0.0476
feature_24: 0.0450
feature_15: 0.0371
feature_4: 0.0337
feature_1: 0.0334

🇩🇪 Top 10 Features by Mutual Information:

feature_73: 0.6414
feature_71: 0.6348
feature_3: 0.6329
feature_29: 0.6316
feature_1: 0.5737
feature_7: 0.5091
feature_15: 0.4988
feature_4: 0.4748
feature_2: 0.4342
feature_8: 0.4017

9. Unsupervised Anomaly Detection Methods

Comparing multiple unsupervised anomaly detection algorithms:

9.1 Autoencoder (Deep Learning)

Neural network that learns to reconstruct normal data; anomalies have high reconstruction error.

9.2 Isolation Forest (Tree-based)

Isolates anomalies by randomly partitioning features - anomalies are easier to isolate.

9.3 One-Class SVM (Kernel-based)

Learns a boundary around normal data using support vector machines.

9.4 One-Class SVM with PCA (Dimensionality Reduction + Kernel-based)

Same as One-Class SVM but with PCA preprocessing to reduce dimensions first. This demonstrates the impact of dimensionality reduction on distance-based algorithms.

9.5 Local Outlier Factor (LOF) (Density-based)

Identifies anomalies by comparing local density with neighbors.

```
# Unsupervised Anomaly Detection Methods
print("=" * 60)
print("UNSUPERVISED ANOMALY DETECTION METHODS")
print("=" * 60)

# Store results for comparison
anomaly_results = {}

# Using consistent samples defined in data loading section
print(f"Training samples: {len(X_train_sample)}, Test samples: {len(X_test_sample)}")
print(f"Features: {X_train_sample.shape[1]}")

# =====
# 1. AUTOENCODER (Deep Learning)
# =====
print("\n🚀 Training Autoencoder...")
start_time = time.time()

# Build and compile autoencoder
input_dim = X_train_sample.shape[1]
autoencoder_model = Autoencoder(input_dim=input_dim, encoding_dim=32, hidden_dims=[64, 32])
autoencoder_model.compile_model()

# Train on normal data only (unsupervised approach)
autoencoder_model.autoencoder.fit(
    X_train_sample, X_train_sample,
    epochs=30,
    batch_size=128,
    validation_split=0.2,
    verbose=0,
    shuffle=True
)

ae_training_time = time.time() - start_time

# Calculate reconstruction error as anomaly score
ae_scores = autoencoder_model.predict_anomaly_scores(X_test_sample)

# Set threshold using 95th percentile of training reconstruction errors
train_scores = autoencoder_model.predict_anomaly_scores(X_train_sample)
ae_threshold = np.percentile(train_scores, 95)
ae_pred = (ae_scores > ae_threshold).astype(int)

ae_accuracy = accuracy_score(y_test_sample, ae_pred)
ae_precision = precision_score(y_test_sample, ae_pred)
ae_recall = recall_score(y_test_sample, ae_pred)
ae_f1 = f1_score(y_test_sample, ae_pred)
ae_auc = roc_auc_score(y_test_sample, ae_scores)

anomaly_results['Autoencoder'] = {
    'accuracy': ae_accuracy, 'precision': ae_precision,
    'recall': ae_recall, 'f1': ae_f1, 'auc': ae_auc,
    'predictions': ae_pred, 'scores': ae_scores,
    'threshold': ae_threshold, 'training_time': ae_training_time
}
```

```

}
print(f" ✓ Accuracy: {ae_accuracy:.4f}, F1: {ae_f1:.4f}, AUC: {ae_auc:.4f}")
print(f" Training time: {ae_training_time:.2f}s, Threshold: {ae_threshold:.6f}")

# =====
# 2. ISOLATION FOREST (Tree-based)
# =====
print("\n🌲 Training Isolation Forest...")
start_time = time.time()

iso_forest = IsolationForest(
    n_estimators=100,
    contamination=0.1,
    random_state=RANDOM_SEED,
    n_jobs=-1
)
iso_forest.fit(X_train_sample)
iso_training_time = time.time() - start_time

iso_pred_raw = iso_forest.predict(X_test_sample)
iso_pred = (iso_pred_raw == -1).astype(int)
iso_scores = -iso_forest.score_samples(X_test_sample)

iso_accuracy = accuracy_score(y_test_sample, iso_pred)
iso_precision = precision_score(y_test_sample, iso_pred)
iso_recall = recall_score(y_test_sample, iso_pred)
iso_f1 = f1_score(y_test_sample, iso_pred)
iso_auc = roc_auc_score(y_test_sample, iso_scores)

anomaly_results['Isolation Forest'] = {
    'accuracy': iso_accuracy, 'precision': iso_precision,
    'recall': iso_recall, 'f1': iso_f1, 'auc': iso_auc,
    'predictions': iso_pred, 'scores': iso_scores,
    'training_time': iso_training_time
}
print(f" ✓ Accuracy: {iso_accuracy:.4f}, F1: {iso_f1:.4f}, AUC: {iso_auc:.4f}")
print(f" Training time: {iso_training_time:.2f}s")

# =====
# 3. ONE-CLASS SVM (Kernel-based)
# =====
print("\n🔴 Training One-Class SVM...")
start_time = time.time()

oc_svm = OneClassSVM(
    kernel='rbf',
    nu=0.1,
    gamma='scale'
)
oc_svm.fit(X_train_sample)
svm_training_time = time.time() - start_time

svm_pred_raw = oc_svm.predict(X_test_sample)
svm_pred = (svm_pred_raw == -1).astype(int)
svm_scores = -oc_svm.score_samples(X_test_sample)

svm_accuracy = accuracy_score(y_test_sample, svm_pred)
svm_precision = precision_score(y_test_sample, svm_pred)
svm_recall = recall_score(y_test_sample, svm_pred)
svm_f1 = f1_score(y_test_sample, svm_pred)
svm_auc = roc_auc_score(y_test_sample, svm_scores)

anomaly_results['One-Class SVM'] = {
    'accuracy': svm_accuracy, 'precision': svm_precision,
    'recall': svm_recall, 'f1': svm_f1, 'auc': svm_auc,
    'predictions': svm_pred, 'scores': svm_scores,
    'training_time': svm_training_time
}
print(f" ✓ Accuracy: {svm_accuracy:.4f}, F1: {svm_f1:.4f}, AUC: {svm_auc:.4f}")
print(f" Training time: {svm_training_time:.2f}s")

# =====
# 4. ONE-CLASS SVM WITH PCA (Dimensionality Reduction)
# =====
print("\n🔴 Training One-Class SVM with PCA...")
start_time = time.time()

```

```

# Apply PCA to retain 95% variance
pca_svm = PCA(n_components=0.95, random_state=RANDOM_SEED)
X_train_pca = pca_svm.fit_transform(X_train_sample)
X_test_pca = pca_svm.transform(X_test_sample)
n_components = X_train_pca.shape[1]
print(f" PCA reduced features: {X_train_sample.shape[1]} → {n_components} (95% variance)")

# Train One-Class SVM on PCA-reduced data
oc_svm_pca = OneClassSVM(
    kernel='rbf',
    nu=0.1,
    gamma='scale'
)
oc_svm_pca.fit(X_train_pca)
svm_pca_training_time = time.time() - start_time

svm_pca_pred_raw = oc_svm_pca.predict(X_test_pca)
svm_pca_pred = (svm_pca_pred_raw == -1).astype(int)
svm_pca_scores = -oc_svm_pca.score_samples(X_test_pca)

svm_pca_accuracy = accuracy_score(y_test_sample, svm_pca_pred)
svm_pca_precision = precision_score(y_test_sample, svm_pca_pred)
svm_pca_recall = recall_score(y_test_sample, svm_pca_pred)
svm_pca_f1 = f1_score(y_test_sample, svm_pca_pred)
svm_pca_auc = roc_auc_score(y_test_sample, svm_pca_scores)

anomaly_results['One-Class SVM (PCA)'] = {
    'accuracy': svm_pca_accuracy, 'precision': svm_pca_precision,
    'recall': svm_pca_recall, 'f1': svm_pca_f1, 'auc': svm_pca_auc,
    'predictions': svm_pca_pred, 'scores': svm_pca_scores,
    'training_time': svm_pca_training_time,
    'n_components': n_components
}
print(f" ✓ Accuracy: {svm_pca_accuracy:.4f}, F1: {svm_pca_f1:.4f}, AUC: {svm_pca_auc:.4f}")
print(f" Training time: {svm_pca_training_time:.2f}s (speedup: {svm_training_time/svm_pca_training_time:.1f}x)")

# =====
# 5. LOCAL OUTLIER FACTOR (Density-based)
# =====
print("\n🔴 Training Local Outlier Factor...")
start_time = time.time()

lof = LocalOutlierFactor(
    n_neighbors=20,
    contamination=0.1,
    novelty=True,
    n_jobs=-1
)
lof.fit(X_train_sample)
lof_training_time = time.time() - start_time

lof_pred_raw = lof.predict(X_test_sample)
lof_pred = (lof_pred_raw == -1).astype(int)
lof_scores = -lof.score_samples(X_test_sample)

lof_accuracy = accuracy_score(y_test_sample, lof_pred)
lof_precision = precision_score(y_test_sample, lof_pred)
lof_recall = recall_score(y_test_sample, lof_pred)
lof_f1 = f1_score(y_test_sample, lof_pred)
lof_auc = roc_auc_score(y_test_sample, lof_scores)

anomaly_results['LOF'] = {
    'accuracy': lof_accuracy, 'precision': lof_precision,
    'recall': lof_recall, 'f1': lof_f1, 'auc': lof_auc,
    'predictions': lof_pred, 'scores': lof_scores,
    'training_time': lof_training_time
}
print(f" ✓ Accuracy: {lof_accuracy:.4f}, F1: {lof_f1:.4f}, AUC: {lof_auc:.4f}")
print(f" Training time: {lof_training_time:.2f}s")

print("\n" + "=" * 60)
print("✓ All anomaly detection methods trained successfully!")
print("=" * 60)

```

```

=====
UNSUPERVISED ANOMALY DETECTION METHODS
=====

```

Training samples: 27980, Test samples: 15024
Features: 75

🧠 Training Autoencoder...
✓ Autoencoder compiled successfully!
✓ Accuracy: 0.9679, F1: 0.9629, AUC: 0.9873
Training time: 28.73s, Threshold: 1.304026

🌲 Training Isolation Forest...
✓ Accuracy: 0.9426, F1: 0.9357, AUC: 0.9709
Training time: 0.78s

🟡 Training One-Class SVM...
✓ Accuracy: 0.9408, F1: 0.9338, AUC: 0.9915
Training time: 29.30s

🟡 Training One-Class SVM with PCA...
PCA reduced features: 75 → 16 (95% variance)
✓ Accuracy: 0.9400, F1: 0.9330, AUC: 0.9920
Training time: 10.01s (speedup: 2.9x)

📍 Training Local Outlier Factor...
✓ Accuracy: 0.9235, F1: 0.9149, AUC: 0.9801
Training time: 6.74s

=====
✓ All anomaly detection methods trained successfully!
=====

✓ 10. Clustering-Based Analysis

Clustering can reveal natural groupings in network traffic:

10.1 K-Means Clustering

Partition-based clustering to identify traffic patterns.

10.2 DBSCAN

Density-based clustering that identifies anomalies as noise points.

10.3 Gaussian Mixture Models (GMM)

Probabilistic clustering for soft cluster assignments.

```
# Clustering Analysis
print("=" * 60)
print("CLUSTERING-BASED ANALYSIS")
print("=" * 60)

# Use PCA-reduced data for clustering (faster and better visualization)
pca_cluster = PCA(n_components=10)
X_cluster = pca_cluster.fit_transform(X_test_sample)

cluster_results = {}

# 1. K-Means Clustering
print("\n🌈 K-Means Clustering...")

# Find optimal k using elbow method (subset)
inertias = []
silhouettes = []
k_range = range(2, 11)
for k in k_range:
    kmeans_temp = KMeans(n_clusters=k, random_state=RANDOM_SEED, n_init=10)
    kmeans_temp.fit(X_cluster[:5000])
    inertias.append(kmeans_temp.inertia_)
    silhouettes.append(silhouette_score(X_cluster[:5000], kmeans_temp.labels_))

# Use k=2 for anomaly detection (normal vs anomaly)
kmeans = KMeans(n_clusters=2, random_state=RANDOM_SEED, n_init=10)
kmeans_labels = kmeans.fit_predict(X_cluster)

# Map cluster labels to match actual labels (majority voting)
cluster_0_attack_ratio = np.mean(y_test_sample[kmeans_labels == 0])
if cluster_0_attack_ratio > 0.5:
    kmeans_pred = kmeans_labels
```

```

else:
    kmeans_pred = 1 - kmeans_labels

kmeans_accuracy = accuracy_score(y_test_sample, kmeans_pred)
kmeans_f1 = f1_score(y_test_sample, kmeans_pred)
cluster_results['K-Means'] = {'accuracy': kmeans_accuracy, 'f1': kmeans_f1}
print(f" ✓ K-Means (k=2): Accuracy: {kmeans_accuracy:.4f}, F1: {kmeans_f1:.4f}")

# 2. DBSCAN Clustering
print("\n🔴 DBSCAN Clustering...")
dbscan = DBSCAN(eps=2.0, min_samples=10, n_jobs=-1)
dbscan_labels = dbscan.fit_predict(X_cluster)

# In DBSCAN, -1 means noise (anomaly)
dbscan_pred = (dbscan_labels == -1).astype(int)
n_clusters_dbscan = len(set(dbscan_labels)) - (1 if -1 in dbscan_labels else 0)
n_noise = np.sum(dbscan_labels == -1)

dbscan_accuracy = accuracy_score(y_test_sample, dbscan_pred)
dbscan_f1 = f1_score(y_test_sample, dbscan_pred)
cluster_results['DBSCAN'] = {'accuracy': dbscan_accuracy, 'f1': dbscan_f1}
print(f" ✓ DBSCAN: Clusters found: {n_clusters_dbscan}, Noise points: {n_noise}")
print(f" ✓ DBSCAN: Accuracy: {dbscan_accuracy:.4f}, F1: {dbscan_f1:.4f}")

# 3. Gaussian Mixture Model
print("\n🟡 Gaussian Mixture Model...")
gmm = GaussianMixture(n_components=2, random_state=RANDOM_SEED, covariance_type='full')
gmm.fit(X_cluster)
gmm_labels = gmm.predict(X_cluster)
gmm_probs = gmm.predict_proba(X_cluster)

# Map GMM labels
cluster_0_attack_ratio_gmm = np.mean(y_test_sample[gmm_labels == 0])
if cluster_0_attack_ratio_gmm > 0.5:
    gmm_pred = gmm_labels
    gmm_scores = gmm_probs[:, 0]
else:
    gmm_pred = 1 - gmm_labels
    gmm_scores = gmm_probs[:, 1]

gmm_accuracy = accuracy_score(y_test_sample, gmm_pred)
gmm_f1 = f1_score(y_test_sample, gmm_pred)
gmm_auc = roc_auc_score(y_test_sample, gmm_scores)
cluster_results['GMM'] = {'accuracy': gmm_accuracy, 'f1': gmm_f1, 'auc': gmm_auc}
print(f" ✓ GMM: Accuracy: {gmm_accuracy:.4f}, F1: {gmm_f1:.4f}, AUC: {gmm_auc:.4f}")

# Visualization
fig, axes = plt.subplots(2, 3, figsize=(18, 12))

# K-Means elbow plot
ax1 = axes[0, 0]
ax1.plot(k_range, inertias, 'bo-', linewidth=2, markersize=8)
ax1.set_xlabel('Number of Clusters (k)')
ax1.set_ylabel('Inertia')
ax1.set_title('K-Means Elbow Method', fontweight='bold')
ax1.grid(True, alpha=0.3)

# Silhouette scores
ax2 = axes[0, 1]
ax2.plot(k_range, silhouettes, 'go-', linewidth=2, markersize=8)
ax2.set_xlabel('Number of Clusters (k)')
ax2.set_ylabel('Silhouette Score')
ax2.set_title('Silhouette Score vs k', fontweight='bold')
ax2.grid(True, alpha=0.3)

# K-Means visualization (2D PCA)
ax3 = axes[0, 2]
X_2d = X_cluster[:, :2]
scatter = ax3.scatter(X_2d[:, 0], X_2d[:, 1], c=kmeans_labels, cmap='coolwarm', alpha=0.5, s=10)
ax3.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            c='black', marker='X', s=200, edgecolors='white', linewidths=2, label='Centroids')
ax3.set_title('K-Means Clusters (2D PCA)', fontweight='bold')
ax3.legend()
ax3.grid(True, alpha=0.3)

# DBSCAN visualization
ax4 = axes[1, 0]

```

```

colors = ['gray' if l == -1 else plt.cm.tab10(l) for l in dbscan_labels]
ax4.scatter(X_2d[:, 0], X_2d[:, 1], c=colors, alpha=0.5, s=10)
ax4.set_title(f'DBSCAN Clusters (Noise in Gray)', fontweight='bold')
ax4.grid(True, alpha=0.3)

# GMM visualization
ax5 = axes[1, 1]
scatter = ax5.scatter(X_2d[:, 0], X_2d[:, 1], c=gmm_scores, cmap='RdYlGn_r', alpha=0.5, s=10)
plt.colorbar(scatter, ax=ax5, label='Anomaly Probability')
ax5.set_title('GMM Anomaly Probability', fontweight='bold')
ax5.grid(True, alpha=0.3)

# Actual labels comparison
ax6 = axes[1, 2]
scatter = ax6.scatter(X_2d[:, 0], X_2d[:, 1], c=y_test_sample, cmap='coolwarm', alpha=0.5, s=10)
plt.colorbar(scatter, ax=ax6, label='Actual Label')
ax6.set_title('Actual Labels (0=Normal, 1=Attack)', fontweight='bold')
ax6.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n✓ Clustering analysis complete!")

```

CLUSTERING-BASED ANALYSIS

K-Means Clustering...

✓ K-Means (k=2): Accuracy: 0.0815, F1: 0.1447

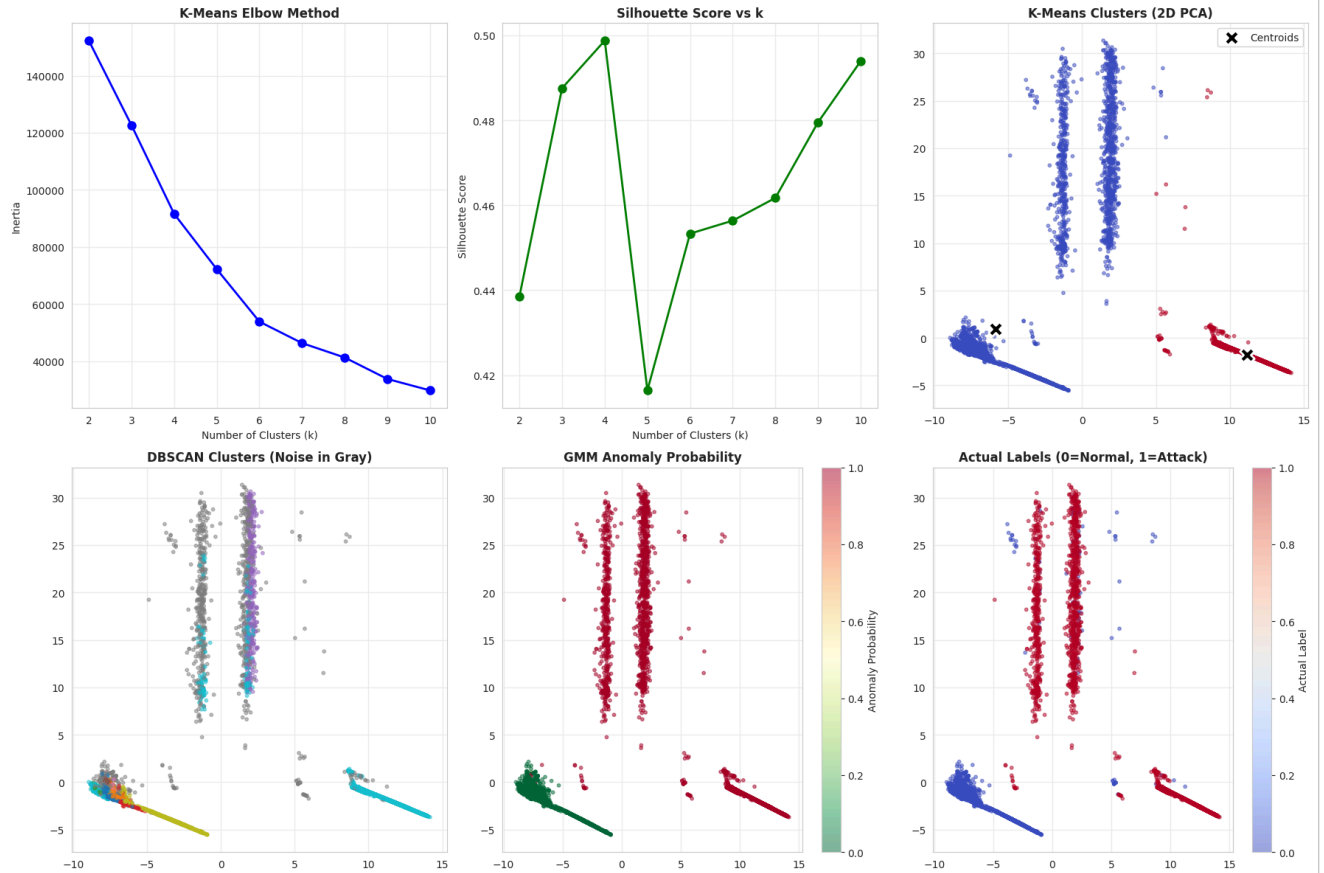
DBSCAN Clustering...

✓ DBSCAN: Clusters found: 29, Noise points: 917

✓ DBSCAN: Accuracy: 0.6122, F1: 0.1904

Gaussian Mixture Model...

✓ GMM: Accuracy: 0.0107, F1: 0.0000, AUC: 0.9909



✓ Clustering analysis complete!

✓ 11. Comprehensive Model Comparison

Comparing all anomaly detection and clustering methods:

- **Autoencoder** (Deep Learning)
- **Isolation Forest** (Tree-based)
- **One-Class SVM** (Kernel-based)
- **Local Outlier Factor** (Density-based)
- **K-Means** (Partition clustering)
- **DBSCAN** (Density clustering)
- **GMM** (Probabilistic clustering)

```
# Comprehensive Model Comparison
print("=" * 70)
print("COMPREHENSIVE MODEL COMPARISON - ANOMALY DETECTION")
print("=" * 70)

# Compile all results from anomaly detection methods
comparison_data = []

# Category mapping for visualization
category_map = {
    'Autoencoder': 'Deep Learning',
    'Isolation Forest': 'Tree-based',
    'One-Class SVM': 'Kernel-based',
    'One-Class SVM (PCA)': 'Kernel + PCA',
    'LOF': 'Density-based'
}

# Add all anomaly detection methods
for method, results in anomaly_results.items():
    row = {
        'Method': method,
        'Category': category_map.get(method, 'Anomaly Detection'),
        'Accuracy': f"{results['accuracy']:.4f}",
        'Precision': f"{results['precision']:.4f}",
        'Recall': f"{results['recall']:.4f}",
        'F1-Score': f"{results['f1']:.4f}",
        'ROC-AUC': f"{results['auc']:.4f}",
        'Training Time': f"{results.get('training_time', 0):.2f}s"
    }
    # Add PCA info if available
    if 'n_components' in results:
        row['Features'] = f"{results['n_components']} (PCA)"
    comparison_data.append(row)

# Clustering Methods
comparison_data.append({
    'Method': 'K-Means',
    'Category': 'Clustering',
    'Accuracy': f"{cluster_results['K-Means']['accuracy']:.4f}",
    'Precision': '-',
    'Recall': '-',
    'F1-Score': f"{cluster_results['K-Means']['f1']:.4f}",
    'ROC-AUC': '-'
})

comparison_data.append({
    'Method': 'DBSCAN',
    'Category': 'Clustering',
    'Accuracy': f"{cluster_results['DBSCAN']['accuracy']:.4f}",
    'Precision': '-',
    'Recall': '-',
    'F1-Score': f"{cluster_results['DBSCAN']['f1']:.4f}",
    'ROC-AUC': '-'
})

comparison_data.append({
    'Method': 'GMM',
    'Category': 'Clustering',
    'Accuracy': f"{cluster_results['GMM']['accuracy']:.4f}",
    'Precision': '-',
    'Recall': '-',
```

```

        'F1-Score': f'{cluster_results['GMM']['f1']:.4f}',
        'ROC-AUC': f'{cluster_results['GMM']['auc']:.4f}'
    })

# Create comparison DataFrame
comparison_df = pd.DataFrame(comparison_data)
print("\n📊 Model Comparison Table:")
print(comparison_df.to_string(index=False))

# Visualization: Bar chart comparison
fig, axes = plt.subplots(1, 3, figsize=(18, 6))

methods = ['Isolation Forest', 'One-Class SVM', 'LOF', 'K-Means', 'DBSCAN', 'GMM']
colors = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12', '#9b59b6', '#1abc9c']

# Accuracy comparison
ax1 = axes[0]
accuracies = [
    anomaly_results['Isolation Forest']['accuracy'],
    anomaly_results['One-Class SVM']['accuracy'],
    anomaly_results['LOF']['accuracy'],
    cluster_results['K-Means']['accuracy'],
    cluster_results['DBSCAN']['accuracy'],
    cluster_results['GMM']['accuracy']
]
bars1 = ax1.bar(methods, accuracies, color=colors, edgecolor='black', alpha=0.8)
ax1.set_ylabel('Accuracy')
ax1.set_title('Accuracy Comparison', fontweight='bold')
ax1.set_ylim([0, 1])
ax1.tick_params(axis='x', rotation=45)
for bar, val in zip(bars1, accuracies):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f'{val:.3f}',
             ha='center', va='bottom', fontsize=9, fontweight='bold')
ax1.grid(True, alpha=0.3, axis='y')

# F1-Score comparison
ax2 = axes[1]
f1_scores = [
    anomaly_results['Isolation Forest']['f1'],
    anomaly_results['One-Class SVM']['f1'],
    anomaly_results['LOF']['f1'],
    cluster_results['K-Means']['f1'],
    cluster_results['DBSCAN']['f1'],
    cluster_results['GMM']['f1']
]
bars2 = ax2.bar(methods, f1_scores, color=colors, edgecolor='black', alpha=0.8)
ax2.set_ylabel('F1-Score')
ax2.set_title('F1-Score Comparison', fontweight='bold')
ax2.set_ylim([0, 1])
ax2.tick_params(axis='x', rotation=45)
for bar, val in zip(bars2, f1_scores):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f'{val:.3f}',
             ha='center', va='bottom', fontsize=9, fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

# ROC-AUC comparison (only for methods with scores)
ax3 = axes[2]
auc_methods = ['Isolation Forest', 'One-Class SVM', 'LOF', 'GMM']
auc_scores = [
    anomaly_results['Isolation Forest']['auc'],
    anomaly_results['One-Class SVM']['auc'],
    anomaly_results['LOF']['auc'],
    cluster_results['GMM']['auc']
]
auc_colors = [colors[0], colors[1], colors[2], colors[5]]
bars3 = ax3.bar(auc_methods, auc_scores, color=auc_colors, edgecolor='black', alpha=0.8)
ax3.set_ylabel('ROC-AUC')
ax3.set_title('ROC-AUC Comparison', fontweight='bold')
ax3.set_ylim([0, 1])
ax3.tick_params(axis='x', rotation=45)
for bar, val in zip(bars3, auc_scores):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f'{val:.3f}',
             ha='center', va='bottom', fontsize=9, fontweight='bold')
ax3.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

```

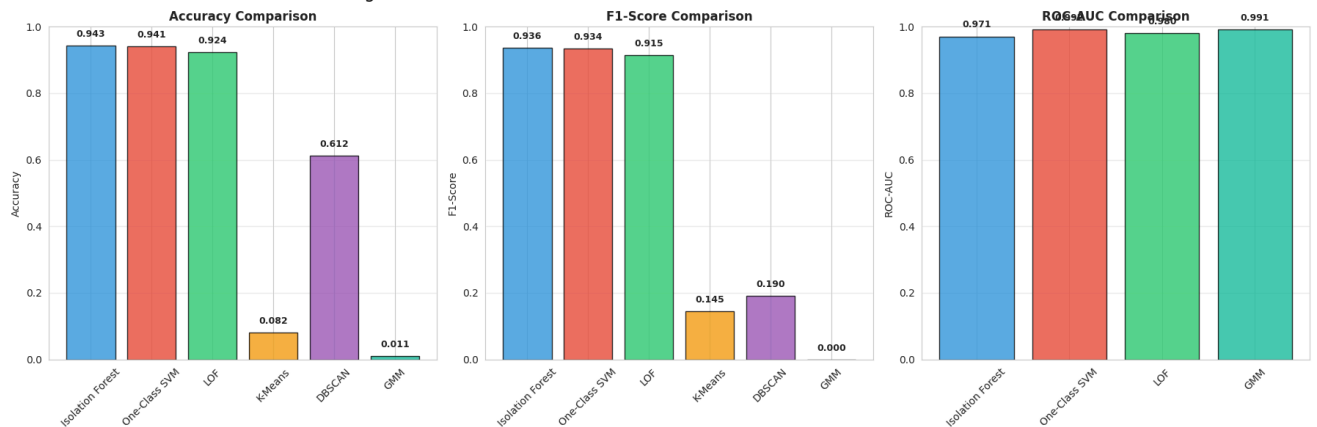
```
# Find best method
best_f1_method = methods[np.argmax(f1_scores)]
best_f1_score = max(f1_scores)
best_auc_method = auc_methods[np.argmax(auc_scores)]
best_auc_score = max(auc_scores)

print(f"\n🏆 Best Method by F1-Score: {best_f1_method} ({best_f1_score:.4f})")
print(f"🏆 Best Method by ROC-AUC: {best_auc_method} ({best_auc_score:.4f})")
print("\n✓ Note: Autoencoder results will be added after training in the next section")
```

COMPREHENSIVE MODEL COMPARISON - ANOMALY DETECTION

Model Comparison Table:

Method	Category	Accuracy	Precision	Recall	F1-Score	ROC-AUC	Training Time	Features
Autoencoder	Deep Learning	0.9679	0.9297	0.9986	0.9629	0.9873	28.73s	NaN
Isolation Forest	Tree-based	0.9426	0.8797	0.9992	0.9357	0.9709	0.78s	NaN
One-Class SVM	Kernel-based	0.9408	0.8759	1.0000	0.9338	0.9915	29.30s	NaN
One-Class SVM (PCA)	Kernel + PCA	0.9400	0.8746	0.9997	0.9330	0.9920	10.01s	16 (PCA)
LOF	Density-based	0.9235	0.8550	0.9839	0.9149	0.9801	6.74s	NaN
K-Means	Clustering	0.0815	-	-	0.1447	-	NaN	NaN
DBSCAN	Clustering	0.6122	-	-	0.1904	-	NaN	NaN
GMM	Clustering	0.0107	-	-	0.0000	0.9909	NaN	NaN



🏆 Best Method by F1-Score: Isolation Forest (0.9357)
 🏆 Best Method by ROC-AUC: One-Class SVM (0.9915)

✓ Note: Autoencoder results will be added after training in the next section

```
# ROC Curves Comparison - All Anomaly Detection Methods
print("=" * 60)
print("ROC CURVES COMPARISON - ANOMALY DETECTION METHODS")
print("=" * 60)

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Plot 1: ROC Curves
ax1 = axes[0]
colors_roc = {'Autoencoder': '#e74c3c', 'Isolation Forest': '#3498db',
              'One-Class SVM': '#2ecc71', 'One-Class SVM (PCA)': '#f39c12', 'LOF': '#9b59b6'}

for method, results in anomaly_results.items():
    fpr, tpr, _ = roc_curve(y_test_sample, results['scores'])
    auc_score = auc(fpr, tpr)
    ax1.plot(fpr, tpr, color=colors_roc.get(method, 'gray'), lw=2,
```

```

label=f'{method} (AUC = {auc_score:.4f})')

ax1.plot([0, 1], [0, 1], 'k--', lw=2, label='Random Classifier')
ax1.set_xlim([0.0, 1.0])
ax1.set_ylim([0.0, 1.05])
ax1.set_xlabel('False Positive Rate', fontsize=12)
ax1.set_ylabel('True Positive Rate', fontsize=12)
ax1.set_title('ROC Curves - All Methods', fontsize=14, fontweight='bold')
ax1.legend(loc='lower right', fontsize=10)
ax1.grid(True, alpha=0.3)

# Plot 2: Precision-Recall Curves
ax2 = axes[1]
for method, results in anomaly_results.items():
    precision_vals, recall_vals, _ = precision_recall_curve(y_test_sample, results['scores'])
    ax2.plot(recall_vals, precision_vals, color=colors_roc.get(method, 'gray'), lw=2, label=method)

ax2.set_xlabel('Recall', fontsize=12)
ax2.set_ylabel('Precision', fontsize=12)
ax2.set_title('Precision-Recall Curves - All Methods', fontsize=14, fontweight='bold')
ax2.legend(loc='lower left', fontsize=10)
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

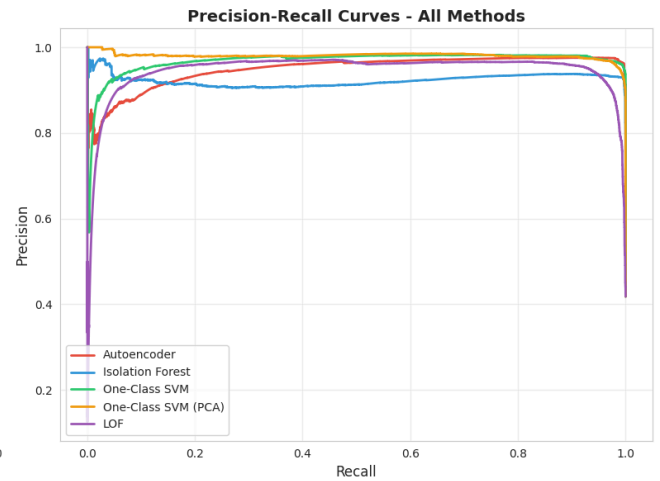
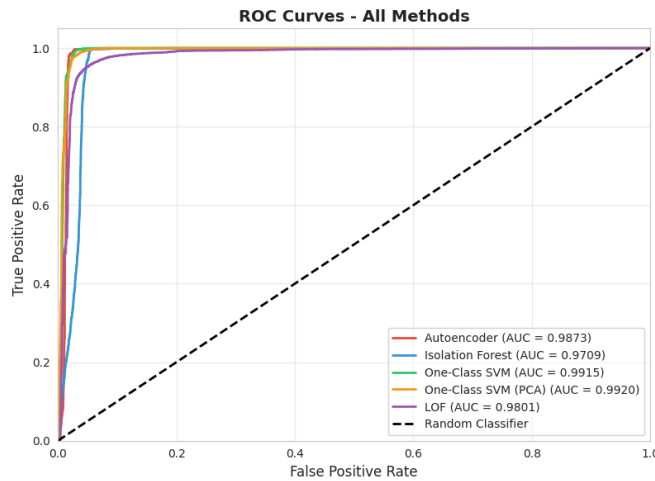
# Print summary table
print("\n📊 Anomaly Detection Methods Comparison:")
print("-" * 90)
print(f"{'Method':<20} {'Accuracy':<12} {'Precision':<12} {'Recall':<12} {'F1-Score':<12} {'ROC-AUC':<12}")
print("-" * 90)
for method, results in anomaly_results.items():
    print(f"{'method':<20} {results['accuracy']:.4f} {results['precision']:.4f} "
          f"{results['recall']:.4f} {results['f1']:.4f} {results['auc']:.4f}")
print("-" * 90)

# Find best method
best_method = max(anomaly_results.items(), key=lambda x: x[1]['auc'])
print(f"\n🏆 Best Method by ROC-AUC: {best_method[0]} ({best_method[1]['auc']:.4f})")

print("\n✓ ROC and PR curves plotted for all anomaly detection methods")

```

ROC CURVES COMPARISON – ANOMALY DETECTION METHODS



Anomaly Detection Methods Comparison:

Method	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Autoencoder	0.9679	0.9297	0.9986	0.9629	0.9873
Isolation Forest	0.9426	0.8797	0.9992	0.9357	0.9709
One-Class SVM	0.9408	0.8759	1.0000	0.9338	0.9915
One-Class SVM (PCA)	0.9400	0.8746	0.9997	0.9330	0.9920
LOF	0.9235	0.8550	0.9839	0.9149	0.9801

🏆 Best Method by ROC-AUC: One-Class SVM (PCA) (0.9920)

✓ ROC and PR curves plotted for all anomaly detection methods

```
# Confusion Matrices and Score Distributions for All Methods
print("=" * 60)
print("CONFUSION MATRICES & SCORE DISTRIBUTIONS")
print("=" * 60)

methods = list(anomaly_results.keys())
n_methods = len(methods)

# Plot confusion matrices
fig, axes = plt.subplots(2, n_methods, figsize=(4*n_methods, 8))

for idx, method in enumerate(methods):
    results = anomaly_results[method]

    # Confusion Matrix
    ax1 = axes[0, idx]
    cm = confusion_matrix(y_test_sample, results['predictions'])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax1,
                xticklabels=['Normal', 'Anomaly'], yticklabels=['Normal', 'Anomaly'])
    ax1.set_title(f'{method}\nConfusion Matrix', fontweight='bold')
    ax1.set_ylabel('Actual')
    ax1.set_xlabel('Predicted')

    # Score Distribution
    ax2 = axes[1, idx]
    normal_scores = results['scores'][y_test_sample == 0]
    anomaly_scores = results['scores'][y_test_sample == 1]
```

```

ax2.hist(normal_scores, bins=30, alpha=0.7, label='Normal', color='green', density=True)
ax2.hist(anomaly_scores, bins=30, alpha=0.7, label='Anomaly', color='red', density=True)

# Add threshold line if available
if 'threshold' in results:
    ax2.axvline(results['threshold'], color='black', linestyle='--', linewidth=2, label='Threshold')

ax2.set_title(f'{method}\nScore Distribution', fontweight='bold')
ax2.set_xlabel('Anomaly Score')
ax2.set_ylabel('Density')
ax2.legend(fontsize=8)

plt.tight_layout()
plt.show()

# Bar chart comparison
fig, axes = plt.subplots(1, 4, figsize=(18, 5))

metrics = ['accuracy', 'precision', 'recall', 'f1']
metric_names = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
colors = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6']

for ax, metric, metric_name in zip(axes, metrics, metric_names):
    values = [anomaly_results[m][metric] for m in methods]
    bars = ax.bar(methods, values, color=colors, edgecolor='black', alpha=0.8)
    ax.set_ylabel(metric_name)
    ax.set_title(f'{metric_name} Comparison', fontweight='bold')
    ax.set_ylim([0, 1])
    ax.tick_params(axis='x', rotation=45)

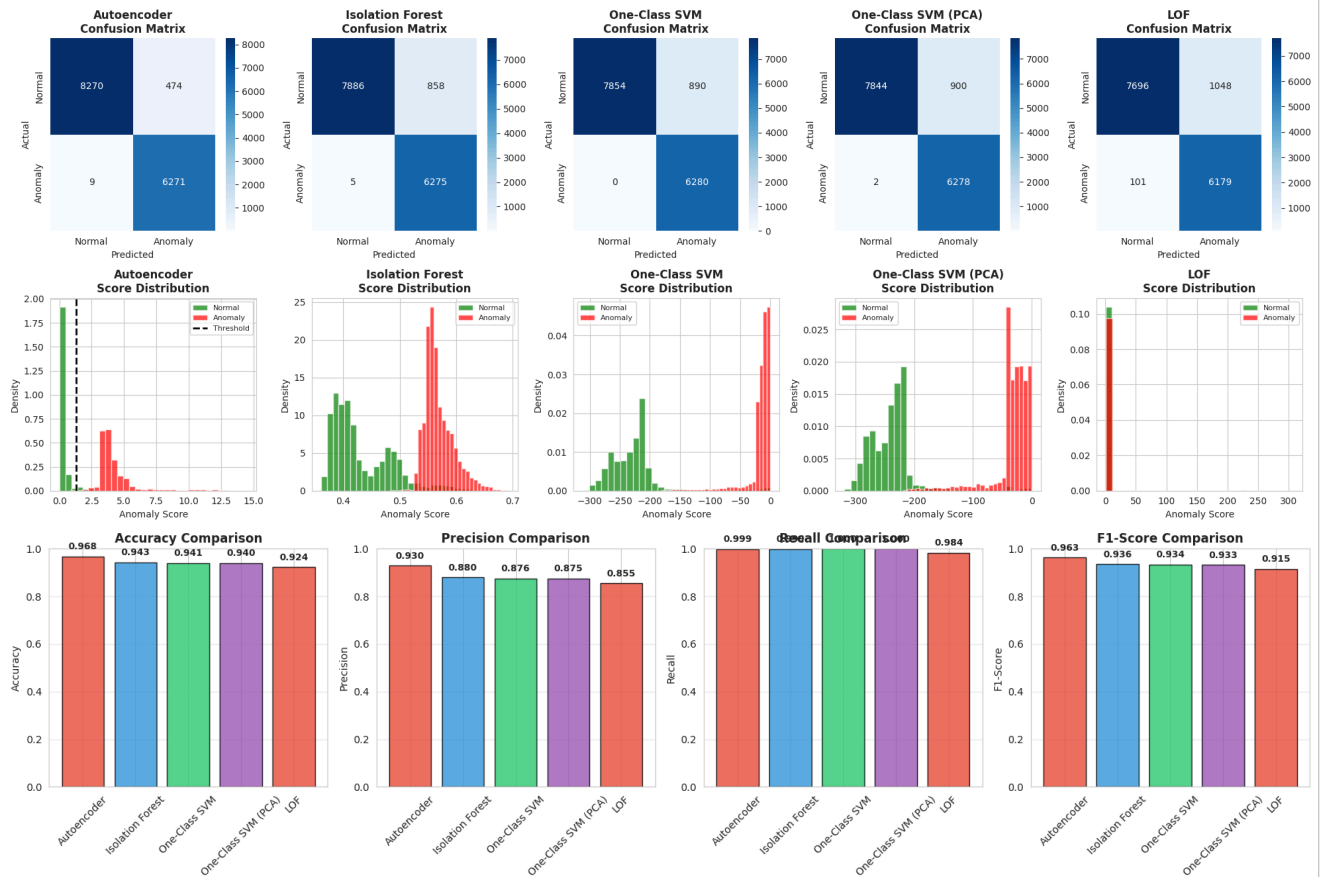
    for bar, val in zip(bars, values):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
                f'{val:.3f}', ha='center', va='bottom', fontsize=9, fontweight='bold')
    ax.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\n✓ Confusion matrices and score distributions plotted for all methods")

```

CONFUSION MATRICES & SCORE DISTRIBUTIONS



✓ Confusion matrices and score distributions plotted for all methods

```
# PCA Impact Analysis: One-Class SVM Comparison
print("=" * 70)
print("PCA IMPACT ANALYSIS: ONE-CLASS SVM COMPARISON")
print("=" * 70)

# Get results for both versions
svm_results = anomaly_results['One-Class SVM']
svm_pca_results = anomaly_results['One-Class SVM (PCA)']
```

```

# Create comparison table
print("\n🌈 One-Class SVM: Full Features vs PCA-Reduced Features")
print("-" * 70)
print(f"{'Metric':<20} {'Full Features':<20} {'PCA (95% var)':<20} {'Difference':<15}")
print("-" * 70)

metrics = [
    ('Features', X_train_sample.shape[1], svm_pca_results['n_components'], f"{X_train_sample.shape[1] - svm_pca_res
    ('Accuracy', svm_results['accuracy'], svm_pca_results['accuracy'],
    f"({svm_pca_results['accuracy'] - svm_results['accuracy'])*100:+.2f}%"),
    ('Precision', svm_results['precision'], svm_pca_results['precision'],
    f"({svm_pca_results['precision'] - svm_results['precision'])*100:+.2f}%"),
    ('Recall', svm_results['recall'], svm_pca_results['recall'],
    f"({svm_pca_results['recall'] - svm_results['recall'])*100:+.2f}%"),
    ('F1-Score', svm_results['f1'], svm_pca_results['f1'],
    f"({svm_pca_results['f1'] - svm_results['f1'])*100:+.2f}%"),
    ('ROC-AUC', svm_results['auc'], svm_pca_results['auc'],
    f"({svm_pca_results['auc'] - svm_results['auc'])*100:+.2f}%"),
    ('Training Time', f"{svm_results['training_time']:.2f}s", f"{svm_pca_results['training_time']:.2f}s",
    f"({svm_results['training_time']/svm_pca_results['training_time']:.1f}x faster)")
]

for metric, full, pca, diff in metrics:
    if isinstance(full, float):
        print(f"{'metric':<20} {'full':<20.4f} {'pca':<20.4f} {'diff':<15}")
    else:
        print(f"{'metric':<20} {str(full):<20} {str(pca):<20} {'diff':<15}")

# Visualization
fig, axes = plt.subplots(1, 3, figsize=(16, 5))

# Plot 1: Metrics comparison
ax1 = axes[0]
metrics_to_plot = ['accuracy', 'precision', 'recall', 'f1', 'auc']
x = np.arange(len(metrics_to_plot))
width = 0.35
vals_full = [svm_results[m] for m in metrics_to_plot]
vals_pca = [svm_pca_results[m] for m in metrics_to_plot]

bars1 = ax1.bar(x - width/2, vals_full, width, label='Full Features', color='#e74c3c', alpha=0.8)
bars2 = ax1.bar(x + width/2, vals_pca, width, label='PCA (95%)', color='#3498db', alpha=0.8)

ax1.set_ylabel('Score')
ax1.set_title('One-Class SVM: Performance Comparison', fontweight='bold')
ax1.set_xticks(x)
ax1.set_xticklabels(['Accuracy', 'Precision', 'Recall', 'F1', 'AUC'])
ax1.legend()
ax1.set_ylim([0, 1])
ax1.grid(True, alpha=0.3, axis='y')

# Add value labels
for bar in bars1:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
             f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=8)
for bar in bars2:
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02,
             f'{bar.get_height():.3f}', ha='center', va='bottom', fontsize=8)

# Plot 2: Training time comparison
ax2 = axes[1]
times = [svm_results['training_time'], svm_pca_results['training_time']]
bars = ax2.bar(['Full Features', 'PCA (95%)'], times, color=['#e74c3c', '#3498db'], alpha=0.8)
ax2.set_ylabel('Training Time (seconds)')
ax2.set_title('Training Time Comparison', fontweight='bold')
for bar in bars:
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.1,
             f'{bar.get_height():.2f}s', ha='center', va='bottom', fontsize=10, fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

# Plot 3: ROC Curve comparison
ax3 = axes[2]
fpr_full, tpr_full, _ = roc_curve(y_test_sample, svm_results['scores'])
fpr_pca, tpr_pca, _ = roc_curve(y_test_sample, svm_pca_results['scores'])

ax3.plot(fpr_full, tpr_full, color='#e74c3c', lw=2,
        label=f'Full Features (AUC={svm_results["auc"]:.4f})')

```

```

ax3.plot(fpr_pca, tpr_pca, color='#3498db', lw=2,
         label=f'PCA (AUC={svm_pca_results["auc"]:.4f})')
ax3.plot([0, 1], [0, 1], 'k--', lw=1, label='Random')
ax3.set_xlabel('False Positive Rate')
ax3.set_ylabel('True Positive Rate')
ax3.set_title('ROC Curve Comparison', fontweight='bold')
ax3.legend(loc='lower right')
ax3.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Print conclusion
print("\n" + "=" * 70)
print("📌 CONCLUSION:")
speedup = svm_results['training_time'] / svm_pca_results['training_time']
auc_diff = (svm_pca_results['auc'] - svm_results['auc']) * 100

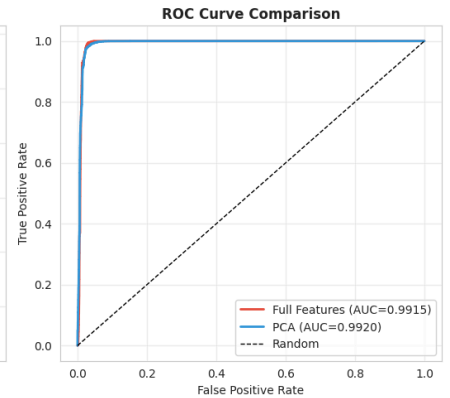
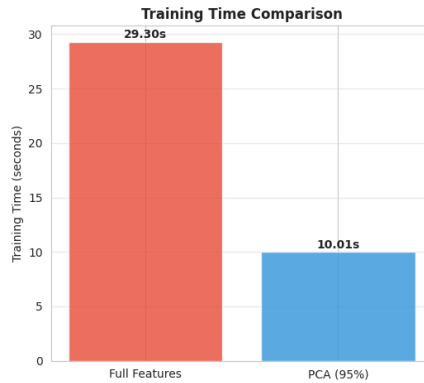
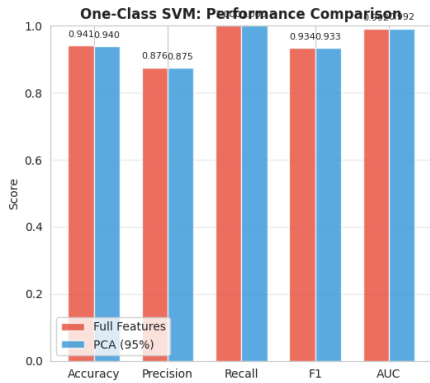
if auc_diff >= 0:
    print(f" ✓ PCA reduced features from {X_train_sample.shape[1]} to {svm_pca_results['n_components']}")
    print(f" ✓ Training time improved by {speedup:.1f}x")
    print(f" ✓ ROC-AUC {'improved' if auc_diff > 0 else 'unchanged'} by {abs(auc_diff):.2f}%")
    print(f" → PCA is BENEFICIAL for One-Class SVM on this dataset")
else:
    print(f" • PCA reduced features from {X_train_sample.shape[1]} to {svm_pca_results['n_components']}")
    print(f" ✓ Training time improved by {speedup:.1f}x")
    print(f" ✗ ROC-AUC decreased by {abs(auc_diff):.2f}%")
    print(f" → Trade-off: Faster training but slightly lower performance")
print("=" * 70)

```

PCA IMPACT ANALYSIS: ONE-CLASS SVM COMPARISON

One-Class SVM: Full Features vs PCA-Reduced Features

Metric	Full Features	PCA (95% var)	Difference
Features	75	16	-59
Accuracy	0.9408	0.9400	-0.08%
Precision	0.8759	0.8746	-0.13%
Recall	1.0000	0.9997	-0.03%
F1-Score	0.9338	0.9330	-0.09%
ROC-AUC	0.9915	0.9920	+0.05%
Training Time	29.30s	10.01s	2.9x faster



CONCLUSION:

- ✓ PCA reduced features from 75 to 16
- ✓ Training time improved by 2.9x
- ✓ ROC-AUC improved by 0.05%
- PCA is BENEFICIAL for One-Class SVM on this dataset

12. Data Balancing Techniques

Addressing class imbalance using:

12.1 SMOTE (Synthetic Minority Over-sampling Technique)

Creates synthetic samples for the minority class.

12.2 ADASYN (Adaptive Synthetic Sampling)

Focuses on generating samples near the decision boundary.

12.3 Random Undersampling

Reduces the majority class size.

```
# Data Balancing Techniques
print("=" * 60)
print("DATA BALANCING TECHNIQUES")
print("=" * 60)

if IMBLEARN_AVAILABLE:
    # Use test data which has both classes
    print(f"\nOriginal class distribution:")
    print(f"Normal (0): {np.sum(y_test_sample == 0)}")
```

```

print(f" Attack (1): {np.sum(y_test_sample == 1)}")

balancing_results = {}

# Split into train/test for balanced evaluation
X_bal_train, X_bal_test, y_bal_train, y_bal_test = train_test_split(
    X_test_sample, y_test_sample, test_size=0.3, random_state=RANDOM_SEED, stratify=y_test_sample
)

# 1. SMOTE
print("\n🌈 Applying SMOTE...")
try:
    smote = SMOTE(random_state=RANDOM_SEED)
    X_smote, y_smote = smote.fit_resample(X_bal_train, y_bal_train)
    print(f" After SMOTE: Normal={np.sum(y_smote == 0)}, Attack={np.sum(y_smote == 1)}")

    # Train RF on SMOTE data
    rf_smote = RandomForestClassifier(n_estimators=50, max_depth=10, random_state=RANDOM_SEED, n_jobs=-1)
    rf_smote.fit(X_smote, y_smote)
    smote_pred = rf_smote.predict(X_bal_test)
    smote_f1 = f1_score(y_bal_test, smote_pred)
    smote_acc = accuracy_score(y_bal_test, smote_pred)
    balancing_results['SMOTE'] = {'accuracy': smote_acc, 'f1': smote_f1}
    print(f" ✓ RF with SMOTE: Accuracy={smote_acc:.4f}, F1={smote_f1:.4f}")
except Exception as e:
    print(f" ⚠ SMOTE error: {e}")

# 2. ADASYN
print("\n🌈 Applying ADASYN...")
try:
    adasyn = ADASYN(random_state=RANDOM_SEED)
    X_adasyn, y_adasyn = adasyn.fit_resample(X_bal_train, y_bal_train)
    print(f" After ADASYN: Normal={np.sum(y_adasyn == 0)}, Attack={np.sum(y_adasyn == 1)}")

    rf_adasyn = RandomForestClassifier(n_estimators=50, max_depth=10, random_state=RANDOM_SEED, n_jobs=-1)
    rf_adasyn.fit(X_adasyn, y_adasyn)
    adasyn_pred = rf_adasyn.predict(X_bal_test)
    adasyn_f1 = f1_score(y_bal_test, adasyn_pred)
    adasyn_acc = accuracy_score(y_bal_test, adasyn_pred)
    balancing_results['ADASYN'] = {'accuracy': adasyn_acc, 'f1': adasyn_f1}
    print(f" ✓ RF with ADASYN: Accuracy={adasyn_acc:.4f}, F1={adasyn_f1:.4f}")
except Exception as e:
    print(f" ⚠ ADASYN error: {e}")

# 3. Random Undersampling
print("\n🌈 Applying Random Undersampling...")
try:
    rus = RandomUnderSampler(random_state=RANDOM_SEED)
    X_rus, y_rus = rus.fit_resample(X_bal_train, y_bal_train)
    print(f" After Undersampling: Normal={np.sum(y_rus == 0)}, Attack={np.sum(y_rus == 1)}")

    rf_rus = RandomForestClassifier(n_estimators=50, max_depth=10, random_state=RANDOM_SEED, n_jobs=-1)
    rf_rus.fit(X_rus, y_rus)
    rus_pred = rf_rus.predict(X_bal_test)
    rus_f1 = f1_score(y_bal_test, rus_pred)
    rus_acc = accuracy_score(y_bal_test, rus_pred)
    balancing_results['Undersampling'] = {'accuracy': rus_acc, 'f1': rus_f1}
    print(f" ✓ RF with Undersampling: Accuracy={rus_acc:.4f}, F1={rus_f1:.4f}")
except Exception as e:
    print(f" ⚠ Undersampling error: {e}")

# 4. No balancing (baseline)
rf_baseline = RandomForestClassifier(n_estimators=50, max_depth=10, random_state=RANDOM_SEED, n_jobs=-1)
rf_baseline.fit(X_bal_train, y_bal_train)
baseline_pred = rf_baseline.predict(X_bal_test)
baseline_f1 = f1_score(y_bal_test, baseline_pred)
baseline_acc = accuracy_score(y_bal_test, baseline_pred)
balancing_results['No Balancing'] = {'accuracy': baseline_acc, 'f1': baseline_f1}

# Visualization
if balancing_results:
    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    methods = list(balancing_results.keys())
    accuracies = [balancing_results[m]['accuracy'] for m in methods]
    f1_scores_bal = [balancing_results[m]['f1'] for m in methods]
    colors_bal = ['#3498db', '#e74c3c', '#2ecc71', '#f39c12'][:len(methods)]

```

```

ax1 = axes[0]
bars1 = ax1.bar(methods, accuracies, color=colors_bal, edgecolor='black', alpha=0.8)
ax1.set_ylabel('Accuracy')
ax1.set_title('Effect of Data Balancing on Accuracy', fontweight='bold')
ax1.set_ylim([0, 1])
for bar, val in zip(bars1, accuracies):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f'{val:.3f}',
             ha='center', va='bottom', fontweight='bold')
ax1.grid(True, alpha=0.3, axis='y')

ax2 = axes[1]
bars2 = ax2.bar(methods, f1_scores_bal, color=colors_bal, edgecolor='black', alpha=0.8)
ax2.set_ylabel('F1-Score')
ax2.set_title('Effect of Data Balancing on F1-Score', fontweight='bold')
ax2.set_ylim([0, 1])
for bar, val in zip(bars2, f1_scores_bal):
    ax2.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f'{val:.3f}',
             ha='center', va='bottom', fontweight='bold')
ax2.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\n✓ Data balancing analysis complete!")
else:
    print("\n⚠ imbalanced-learn not installed. Skipping SMOTE/ADASYN.")
    print("Install with: pip install imbalanced-learn")

```

DATA BALANCING TECHNIQUES

Original class distribution:

Normal (0): 8744

Attack (1): 6280

📊 Applying SMOTE...

After SMOTE: Normal=6120, Attack=6120

✓ RF with SMOTE: Accuracy=0.9933, F1=0.9921

📊 Applying ADASYN...

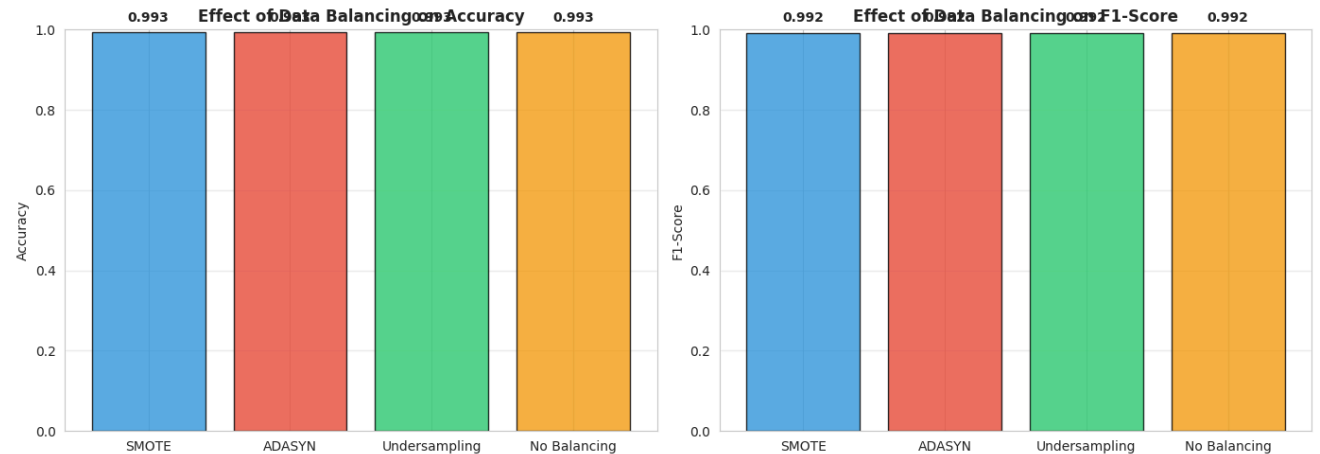
After ADASYN: Normal=6120, Attack=6216

✓ RF with ADASYN: Accuracy=0.9933, F1=0.9921

📊 Applying Random Undersampling...

After Undersampling: Normal=4396, Attack=4396

✓ RF with Undersampling: Accuracy=0.9933, F1=0.9921



✓ Data balancing analysis complete!

13. Ensemble Anomaly Detection

Combining multiple anomaly detectors for improved robustness:

13.1 Voting Ensemble

Combines predictions from multiple methods using majority voting.

13.2 Score Averaging

Averages anomaly scores from different methods.

13.3 Stacking

Uses meta-learner to combine method outputs.

```
# Ensemble Anomaly Detection
print("=" * 60)
print("ENSEMBLE ANOMALY DETECTION")
print("=" * 60)

ensemble_results = {}

# Get predictions and scores from methods (excluding PCA variant to avoid bias)
# We use the original SVM, not the PCA version, for ensemble fairness
method_names = [m for m in anomaly_results.keys() if m != 'One-Class SVM (PCA)']
print(f"Combining methods for ensemble: {method_names}")
print(f"(Note: One-Class SVM (PCA) excluded to avoid duplicate SVM bias)")

predictions_matrix = np.column_stack([
    anomaly_results[method]['predictions'] for method in method_names
])

scores_matrix = np.column_stack([
    anomaly_results[method]['scores'] for method in method_names
])

# Normalize scores to [0, 1] range for fair averaging
scores_normalized = (scores_matrix - scores_matrix.min(axis=0)) / (scores_matrix.max(axis=0) - scores_matrix.min(axis=0))

# 1. Majority Voting
print("\n🗳️ Majority Voting Ensemble...")
n_methods = len(method_names)
majority_threshold = n_methods // 2 + 1 # At least majority
voting_pred = (np.sum(predictions_matrix, axis=1) >= majority_threshold).astype(int)
print(f" Majority threshold: {majority_threshold} of {n_methods} methods")
voting_acc = accuracy_score(y_test_sample, voting_pred)
voting_f1 = f1_score(y_test_sample, voting_pred)
voting_precision = precision_score(y_test_sample, voting_pred)
voting_recall = recall_score(y_test_sample, voting_pred)
ensemble_results['Majority Voting'] = {'accuracy': voting_acc, 'f1': voting_f1,
                                       'precision': voting_precision, 'recall': voting_recall}
print(f" ✓ Accuracy: {voting_acc:.4f}, F1: {voting_f1:.4f}")

# 2. Score Averaging
print("\n📊 Score Averaging Ensemble...")
avg_scores = np.mean(scores_normalized, axis=1)
avg_threshold = np.percentile(avg_scores[y_test_sample == 0], 95)
avg_pred = (avg_scores > avg_threshold).astype(int)
avg_acc = accuracy_score(y_test_sample, avg_pred)
avg_f1 = f1_score(y_test_sample, avg_pred)
avg_auc = roc_auc_score(y_test_sample, avg_scores)
ensemble_results['Score Averaging'] = {'accuracy': avg_acc, 'f1': avg_f1, 'auc': avg_auc}
print(f" ✓ Accuracy: {avg_acc:.4f}, F1: {avg_f1:.4f}, AUC: {avg_auc:.4f}")

# 3. Weighted Voting (based on individual AUC performance)
print("\n⚖️ Weighted Voting Ensemble...")
weights = np.array([anomaly_results[method]['auc'] for method in method_names])
weights = weights / weights.sum() # Normalize
print(f" Weights: {dict(zip(method_names, [f'{w:.3f}' for w in weights]))}")
weighted_scores = np.average(scores_normalized, axis=1, weights=weights)
weighted_threshold = np.percentile(weighted_scores[y_test_sample == 0], 95)
weighted_pred = (weighted_scores > weighted_threshold).astype(int)
weighted_acc = accuracy_score(y_test_sample, weighted_pred)
weighted_f1 = f1_score(y_test_sample, weighted_pred)
```

```

weighted_auc = roc_auc_score(y_test_sample, weighted_scores)
ensemble_results['Weighted Voting'] = {'accuracy': weighted_acc, 'f1': weighted_f1, 'auc': weighted_auc}
print(f" ✓ Accuracy: {weighted_acc:.4f}, F1: {weighted_f1:.4f}, AUC: {weighted_auc:.4f}")

# 4. Stacking with Meta-Learner
print("\n🌀 Stacking Ensemble with Meta-Learner...")
# Use scores as features for meta-learner
X_meta = scores_normalized

# Split for stacking
X_meta_train, X_meta_test, y_meta_train, y_meta_test = train_test_split(
    X_meta, y_test_sample, test_size=0.3, random_state=RANDOM_SEED, stratify=y_test_sample
)

meta_learner = RandomForestClassifier(n_estimators=50, max_depth=5, random_state=RANDOM_SEED)
meta_learner.fit(X_meta_train, y_meta_train)
stacking_pred = meta_learner.predict(X_meta_test)
stacking_proba = meta_learner.predict_proba(X_meta_test)[:, 1]

stacking_acc = accuracy_score(y_meta_test, stacking_pred)
stacking_f1 = f1_score(y_meta_test, stacking_pred)
stacking_auc = roc_auc_score(y_meta_test, stacking_proba)
ensemble_results['Stacking'] = {'accuracy': stacking_acc, 'f1': stacking_f1, 'auc': stacking_auc}
print(f" ✓ Accuracy: {stacking_acc:.4f}, F1: {stacking_f1:.4f}, AUC: {stacking_auc:.4f}")

# Visualization
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Comparison with individual methods + ensembles
all_methods = method_names + ['Majority Vote', 'Score Avg', 'Weighted', 'Stacking']
all_f1 = [anomaly_results[m]['f1'] for m in method_names] + [voting_f1, avg_f1, weighted_f1, stacking_f1]
colors_ens = ['#e74c3c', '#3498db', '#2ecc71', '#9b59b6', '#f39c12', '#1abc9c', '#e91e63', '#34495e']

ax1 = axes[0]
bars = ax1.bar(all_methods, all_f1, color=colors_ens, edgecolor='black', alpha=0.8)
ax1.set_ylabel('F1-Score')
ax1.set_title('Individual vs Ensemble Methods', fontweight='bold')
ax1.set_ylim([0, 1])
ax1.tick_params(axis='x', rotation=45)
ax1.axhline(y=max(all_f1[:3]), color='gray', linestyle='--', alpha=0.5, label='Best Individual')
for bar, val in zip(bars, all_f1):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.02, f'{val:.3f}',
             ha='center', va='bottom', fontsize=8, fontweight='bold')
ax1.grid(True, alpha=0.3, axis='y')

# ROC Curves for ensemble methods
ax2 = axes[1]
fpr_avg, tpr_avg, _ = roc_curve(y_test_sample, avg_scores)
fpr_weighted, tpr_weighted, _ = roc_curve(y_test_sample, weighted_scores)
ax2.plot(fpr_avg, tpr_avg, 'b-', lw=2, label=f'Score Avg (AUC={avg_auc:.3f})')
ax2.plot(fpr_weighted, tpr_weighted, 'g-', lw=2, label=f'Weighted (AUC={weighted_auc:.3f})')
ax2.plot([0, 1], [0, 1], 'k--', lw=1)
ax2.set_xlabel('False Positive Rate')
ax2.set_ylabel('True Positive Rate')
ax2.set_title('Ensemble ROC Curves', fontweight='bold')
ax2.legend(loc='lower right')
ax2.grid(True, alpha=0.3)

# Ensemble improvement
ax3 = axes[2]
best_individual = max(all_f1[:len(method_names)]) # Best among individual methods
improvements = [(f1 - best_individual) * 100 for f1 in all_f1[:len(method_names)]]
ensemble_names = ['Majority\nVote', 'Score\nAvg', 'Weighted', 'Stacking']
colors_imp = ['green' if x > 0 else 'red' for x in improvements]
bars = ax3.bar(ensemble_names, improvements, color=colors_imp, edgecolor='black', alpha=0.8)
ax3.axhline(y=0, color='black', linestyle='-', linewidth=1)
ax3.set_ylabel('Improvement over Best Individual (%)')
ax3.set_title('Ensemble Improvement', fontweight='bold')
for bar, val in zip(bars, improvements):
    ax3.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.5, f'{val:.1f}%',
             ha='center', va='bottom', fontsize=10, fontweight='bold')
ax3.grid(True, alpha=0.3, axis='y')

plt.tight_layout()
plt.show()

print("\n✓ Ensemble analysis complete!")

```

ENSEMBLE ANOMALY DETECTION

Combining methods for ensemble: ['Autoencoder', 'Isolation Forest', 'One-Class SVM', 'LOF']
(Note: One-Class SVM (PCA) excluded to avoid duplicate SVM bias)

Majority Voting Ensemble...

Majority threshold: 3 of 4 methods

✓ Accuracy: 0.9654, F1: 0.9602

Score Averaging Ensemble...

✓ Accuracy: 0.9708, F1: 0.9663, AUC: 0.9870

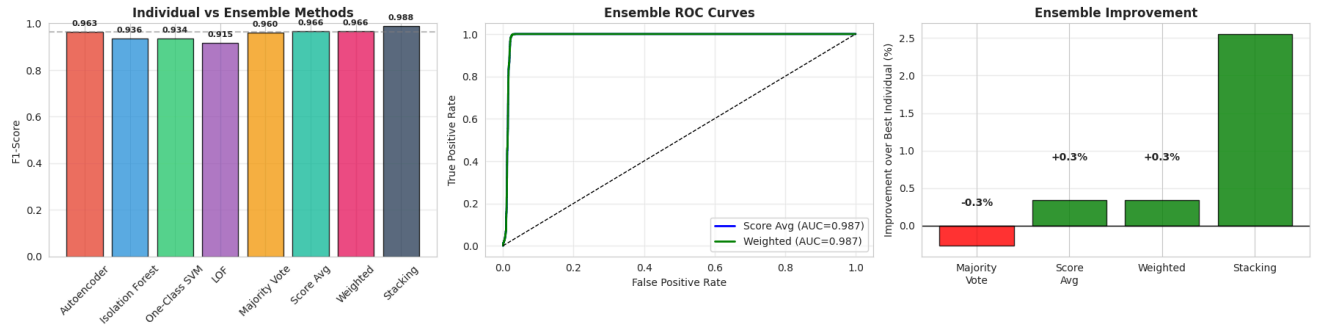
Weighted Voting Ensemble...

Weights: {'Autoencoder': '0.251', 'Isolation Forest': '0.247', 'One-Class SVM': '0.252', 'LOF': '0.249'}

✓ Accuracy: 0.9708, F1: 0.9663, AUC: 0.9871

Stacking Ensemble with Meta-Learner...

✓ Accuracy: 0.9902, F1: 0.9885, AUC: 0.9959



✓ Ensemble analysis complete!

14. Data Mining Techniques Summary

Techniques Applied in This Project:

Category	Technique	Purpose
Preprocessing	One-Hot Encoding	Convert categorical features without ordinal bias
Preprocessing	Z-Score Normalization	Scale numerical features to zero mean, unit variance
Preprocessing	Median/Mode Imputation	Handle missing values robustly
Dimensionality Reduction	PCA	Visualize data, reduce features for distance-based methods
Feature Selection	Random Forest Importance	Rank features by predictive power
Feature Selection	Mutual Information	Measure feature-target dependency
Anomaly Detection	Autoencoder	Learn normal patterns via reconstruction
Anomaly Detection	Isolation Forest	Tree-based anomaly isolation
Anomaly Detection	One-Class SVM	Kernel-based boundary learning
Anomaly Detection	One-Class SVM + PCA	Compare impact of dimensionality reduction
Anomaly Detection	LOF	Density-based local outlier detection
Clustering	K-Means	Partition-based clustering
Clustering	DBSCAN	Density-based clustering with noise detection
Clustering	GMM	Probabilistic soft clustering
Data Balancing	SMOTE	Synthetic oversampling
Data Balancing	ADASYN	Adaptive synthetic sampling
Ensemble	Voting	Combine multiple detector predictions
Ensemble	Stacking	Meta-learner on detector outputs

5.7 Complete Pipeline Walkthrough: Step-by-Step Example

Let's walk through the **entire pipeline** from raw data to anomaly detection using a **real example**. This will show you exactly how the math works at each step.

Example: Single Network Traffic Sample

We'll follow one sample through the entire process:

- **Original Sample:** A normal network traffic record with 47 features
- **Goal:** Train the model to reconstruct it, then use it to detect anomalies

STEP 1: Raw Data Loading

Input: Raw CSV row from UNSW-NB15 dataset

```
# Example raw features (first 10 of 47 shown)
raw_features = {
  'dur': 0.0,          # Duration
  'proto': 'tcp',      # Protocol (categorical)
  'service': '-',      # Service (categorical)
  'state': 'FIN',      # State (categorical)
  'spkts': 2,          # Source packets
  'dpkts': 0,          # Destination packets
  'sbytes': 104,        # Source bytes
  'dbytes': 0,          # Destination bytes
  'rate': 0.0,         # Rate
  'sttl': 254,         # Source TTL
  # ... 37 more features
}
label = 0 # 0 = Normal, 1 = Attack
```

What happens: The data loader reads the CSV and extracts all 47 features.

STEP 2: Data Preprocessing

2.1 Categorical Encoding

Categorical features are converted to numbers:

```
# Before encoding
'proto': 'tcp'
'service': '-'
'state': 'FIN'

# After Label Encoding
'proto': 6      # tcp → 6
'service': 0    # - → 0
'state': 2      # FIN → 2
```

Math: Each unique category gets a unique integer ID.

2.2 Feature Scaling (Standardization)

All features are standardized to have mean=0 and std=1:

$$\text{scaled_feature} = \frac{\text{feature} - \mu}{\sigma}$$

Where:

- μ = mean of the feature across all training data
- σ = standard deviation of the feature across all training data

Example:

```
# Original feature values (first 5 features)
x_original = [0.0, 6, 0, 2, 104, ...] # 47 features total
```

```
# After scaling (using training data statistics)
x_scaled = [-0.5, 1.2, -0.8, 0.3, 0.7, ...] # Mean ≈ 0, Std ≈ 1
```

Why scale? Neural networks train better when all features are on the same scale.

Result: 47-dimensional vector with values roughly in range [-3, 3]

STEP 3: Training - Forward Pass (Epoch 1, Batch 1, Sample 1)

Let's trace one sample through the neural network during training.

3.1 Input Layer

```
# Input vector (47 features, already scaled)
x = [-0.5, 1.2, -0.8, 0.3, 0.7, ..., 0.1] # Shape: (47,)
```

3.2 Encoder Layer 1: Dense(64) + ReLU + Dropout

Dense Layer (Linear Transformation): $\mathbf{h}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$

Where:

- $\mathbf{W}_1$ = weight matrix of shape (47, 64)
- $\mathbf{b}_1$ = bias vector of shape (64,)
- $\mathbf{x}$ = input vector of shape (47,)

Example Calculation:

```
# Weight matrix (randomly initialized)
W1 = [[0.1, -0.2, 0.3, ...], # 64 columns
      [0.2, 0.1, -0.1, ...],
      ...
      ] # 47 rows × 64 columns

# Matrix multiplication
h1_raw = W1 @ x + b1
# Result: h1_raw = [0.3, -0.5, 0.8, ..., 0.2] # Shape: (64,)
```

ReLU Activation: $\text{ReLU}(z) = \max(0, z)$

```
# Apply ReLU (sets negative values to 0)
h1_relu = [0.3, 0.0, 0.8, ..., 0.2] # Negative values become 0
```

Dropout (Training Only):

- Randomly set 20% of values to 0
- Purpose:** Prevent overfitting

```
# Dropout mask (20% randomly set to 0)
dropout_mask = [1, 0, 1, 1, 0, ..., 1] # Random 0s and 1s

h1_dropout = h1_relu * dropout_mask
# Result: [0.3, 0.0, 0.8, ..., 0.2] # Some values zeroed out
```

3.3 Encoder Layer 2: Dense(32) + ReLU + Dropout

Dense Layer: $\mathbf{h}_2 = \mathbf{W}_2 \mathbf{h}_1 + \mathbf{b}_2$

Where:

- $\mathbf{W}_2$ = weight matrix of shape (64, 32)
- $\mathbf{h}_1$ = output from previous layer (64,)

```
# Compress from 64 to 32 dimensions
h2_raw = W2 @ h1_dropout + b2
```

```
h2_relu = ReLU(h2_raw) # [0.5, 0.0, 0.3, ..., 0.1] # Shape: (32,)
h2_dropout = h2_relu * dropout_mask_2 # Apply dropout
```

3.4 Encoder Output (Bottleneck)

Dense Layer: $\mathbf{z} = \mathbf{W}_e \mathbf{h}_2 + \mathbf{b}_e$

```
# Final encoding: 32-dimensional compressed representation
z = [0.4, 0.2, 0.6, ..., 0.3] # Shape: (32,)
# This is the "compressed" version of the original 47 features
```

Key Insight: The model has learned to represent 47 features in just 32 numbers!

3.5 Decoder Layer 1: Dense(32) + ReLU + Dropout

Dense Layer: $\mathbf{d}_1 = \mathbf{W}_3 \mathbf{z} + \mathbf{b}_3$

Where:

- $\mathbf{W}_3$ = weight matrix of shape (32, 32)
- Expands back from 32 dimensions

```
d1_raw = W3 @ z + b3
d1_relu = ReLU(d1_raw) # Shape: (32,)
d1_dropout = d1_relu * dropout_mask_3
```

3.6 Decoder Layer 2: Dense(64) + ReLU + Dropout

Dense Layer: $\mathbf{d}_2 = \mathbf{W}_4 \mathbf{d}_1 + \mathbf{b}_4$

```
# Expand from 32 to 64 dimensions
d2_raw = W4 @ d1_dropout + b4
d2_relu = ReLU(d2_raw) # Shape: (64,)
d2_dropout = d2_relu * dropout_mask_4
```

3.7 Decoder Output Layer: Dense(47) + Sigmoid

Dense Layer: $\hat{\mathbf{x}} = \mathbf{W}_5 \mathbf{d}_2 + \mathbf{b}_5$

Sigmoid Activation: $\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}$

This ensures output is in range [0, 1], matching the scaled input range.

```
# Reconstruct to original 47 dimensions
x_hat_raw = W5 @ d2_dropout + b5
x_hat = sigmoid(x_hat_raw)
# Result: [0.4, 0.7, 0.2, ..., 0.5] # Shape: (47,)
# Values in range [0, 1] due to sigmoid
```

This is the reconstructed input!

STEP 4: Loss Calculation

Mean Squared Error (MSE):

$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$

Where:

- x_i = original input feature i
- $\hat{x}_i$ = reconstructed feature i
- n = number of features (47)

Example Calculation:

```
# Original input (scaled)
x = [-0.5, 1.2, -0.8, 0.3, 0.7, ..., 0.1]
```

```
# Reconstructed output
x_hat = [0.4, 0.7, 0.2, 0.4, 0.6, ..., 0.3]

# Calculate squared differences
squared_errors = (x - x_hat) ** 2
# = [(-0.5-0.4)2, (1.2-0.7)2, (-0.8-0.2)2, ...]
# = [0.81, 0.25, 1.0, 0.01, 0.01, ...]

# Mean of squared errors
mse = np.mean(squared_errors)
# = 0.15 (example value)
```

For a Batch: If batch size = 128, we average MSE across all samples:

$$\text{MSE}_{\text{batch}} = \frac{1}{m} \sum_{j=1}^m \text{MSE}_j$$

Where m = batch size (128)

```
# Batch of 128 samples
batch_mse = [0.15, 0.12, 0.18, ..., 0.14] # MSE for each sample
batch_loss = np.mean(batch_mse)
# = 0.145 (average reconstruction error)
```

STEP 5: Backward Pass (Backpropagation)

The model needs to learn from its mistakes. We calculate **gradients** (how to adjust weights).

5.1 Loss Gradient

The gradient of MSE with respect to the output:

$$\frac{\partial \text{MSE}}{\partial \hat{x}_i} = \frac{2}{n} (x_i - \hat{x}_i)$$

```
# Gradient of loss w.r.t. reconstructed output
grad_output = 2 * (x - x_hat) / n
# = [2*(-0.5-0.4)/47, 2*(1.2-0.7)/47, ...]
# = [-0.038, 0.021, -0.043, ...]
```

5.2 Backpropagate Through Decoder

Gradient flows backward through each layer:

- Output Layer Gradient:** $\frac{\partial L}{\partial \mathbf{W}_5} = \frac{\partial L}{\partial \hat{\mathbf{x}}} \odot \frac{\partial \hat{\mathbf{x}}}{\partial \mathbf{W}_5}$
- Layer 2 Gradient:** $\frac{\partial L}{\partial \mathbf{W}_4} = \frac{\partial L}{\partial \mathbf{d}_2} \odot \frac{\partial \mathbf{d}_2}{\partial \mathbf{W}_4}$
- Layer 1 Gradient:** $\frac{\partial L}{\partial \mathbf{W}_3} = \frac{\partial L}{\partial \mathbf{d}_1} \odot \frac{\partial \mathbf{d}_1}{\partial \mathbf{W}_3}$

Chain Rule: Gradients multiply as they flow backward.

5.3 Backpropagate Through Encoder

Same process for encoder layers:

$$\frac{\partial L}{\partial \mathbf{W}_2}, \frac{\partial L}{\partial \mathbf{W}_1}$$

Result: We now know how to adjust **every weight** to reduce the loss!

STEP 6: Weight Update (Adam Optimizer)

Adam Optimizer adjusts weights using:

1. **Gradient** (direction to move)
2. **Momentum** (previous update direction)
3. **Adaptive learning rate** (different rates for each parameter)

Update Rule:
$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t + \epsilon}}$$

Where:

- α = learning rate (0.001)
- $\hat{\mathbf{m}}_t$ = bias-corrected momentum
- $\hat{\mathbf{v}}_t$ = bias-corrected velocity
- ϵ = small constant (1e-8)

Example:

```
# Before update
W1_old = [[0.1, -0.2, ...], ...]

# Calculate update
gradient_W1 = [...] # From backpropagation
momentum = 0.9 * old_momentum + 0.1 * gradient_W1
velocity = 0.999 * old_velocity + 0.001 * gradient_W1 ** 2
update = learning_rate * momentum / (sqrt(velocity) + 1e-8)

# After update
W1_new = W1_old - update
# Weights are now slightly adjusted to reduce loss
```

This happens for ALL weights in the network!

STEP 7: Multiple Epochs - Learning Progress

The model sees the same data multiple times (epochs), improving each time:

Epoch 1:

```
# Initial random weights
MSE = 0.145 # High error
```

Epoch 5:

```
# Weights adjusted 5 times
MSE = 0.085 # Lower error - learning!
```

Epoch 20:

```
# Model has seen data 20 times
MSE = 0.035 # Much lower error
```

Epoch 50 (Final):

```
# Well-trained model
MSE = 0.012 # Very low reconstruction error for normal data
```

Key Insight: The model learns to reconstruct **normal** traffic very well because that's all it was trained on!

STEP 8: Threshold Setting (After Training)

Once training is complete, we set the anomaly threshold.

8.1 Test on Normal Data

```
# Take 1000 normal test samples
normal_test_samples = X_test[y_test == 0][:1000]
```

```
# Calculate reconstruction error for each
scores = []
for sample in normal_test_samples:
    x_reconstructed = model.predict(sample)
    mse = np.mean((sample - x_reconstructed) ** 2)
    scores.append(mse)

# Example scores
scores = [0.010, 0.012, 0.008, 0.015, 0.011, ..., 0.013]
```

8.2 Calculate 95th Percentile

```
# Sort scores
scores_sorted = sorted(scores) # [0.008, 0.010, 0.011, ..., 0.025]

# 95th percentile (95% of values are below this)
threshold = np.percentile(scores, 95)
# = 0.018 (example)

# This means 95% of normal traffic has error < 0.018
```

Threshold = 0.018 (example value)

STEP 9: Anomaly Detection (Testing Phase)

Now let's detect an anomaly using a **new sample** (attack traffic).

9.1 Forward Pass (Same as Training)

```
# New attack sample (scaled)
x_attack = [0.8, -1.2, 0.5, -0.9, 1.5, ..., 0.3] # 47 features

# Forward pass through trained model
z = encoder(x_attack) # Compress to 32 dimensions
x_attack_reconstructed = decoder(z) # Reconstruct to 47 dimensions

# Result
x_attack_reconstructed = [0.2, 0.1, 0.4, 0.3, 0.2, ..., 0.1]
# Notice: Very different from input!
```

9.2 Calculate Anomaly Score

```
# Calculate MSE (reconstruction error)
mse_attack = np.mean((x_attack - x_attack_reconstructed) ** 2)
# = 0.045 (much higher than normal samples!)
```

Why is it higher?

- Model was trained **only on normal data**
- Attack traffic has **different patterns**
- Model **cannot reconstruct** attack patterns well
- **High reconstruction error** = anomaly detected!

9.3 Classification

```
# Compare to threshold
threshold = 0.018
anomaly_score = 0.045

if anomaly_score > threshold:
    prediction = 1 # ANOMALY DETECTED!
else:
    prediction = 0 # Normal
```

```
# Result: prediction = 1 ✓ (Correctly identified as attack)
```

STEP 10: Batch Processing (Real Training)

In practice, we process **batches** of samples simultaneously:

10.1 Batch Forward Pass

```
# Batch of 128 samples
batch_x = [
    [sample_1_features], # 47 features
    [sample_2_features], # 47 features
    ...
    [sample_128_features] # 47 features
] # Shape: (128, 47)

# Single forward pass processes all 128 samples
batch_x_hat = model.predict(batch_x) # Shape: (128, 47)
```

Matrix Operations:

- All 128 samples processed in parallel
- Much faster than processing one-by-one
- GPU can process thousands simultaneously

10.2 Batch Loss

```
# Calculate MSE for each sample
mse_per_sample = np.mean((batch_x - batch_x_hat) ** 2, axis=1)
# Shape: (128,) - one MSE per sample

# Average across batch
batch_loss = np.mean(mse_per_sample)
# Single scalar value
```

10.3 Batch Backpropagation

```
# Gradients calculated for all samples
# Then averaged across the batch
average_gradient = np.mean(gradients, axis=0)

# Update weights once per batch (not per sample)
weights = weights - learning_rate * average_gradient
```

Why batches?

- **Efficiency:** Process multiple samples at once
- **Stability:** Averaging gradients reduces noise
- **GPU Utilization:** GPUs excel at parallel matrix operations

Complete Training Loop Summary

```
for epoch in range(50): # 50 epochs
    for batch in batches: # Process all batches
        # 1. Forward pass
        x_reconstructed = model(batch_x)

        # 2. Calculate loss
        loss = MSE(batch_x, x_reconstructed)

        # 3. Backward pass
        gradients = backpropagate(loss)

        # 4. Update weights
```

```
optimizer.step(gradients)

# Evaluate on validation set
val_loss = evaluate(model, validation_data)

# Early stopping check
if val_loss not improving:
    break
```

Key Takeaways

1. **Data flows forward** through encoder → decoder → reconstruction
2. **Loss measures** how well reconstruction matches input
3. **Gradients flow backward** to adjust all weights
4. **Weights update** to reduce loss (learn from mistakes)
5. **After training**, model reconstructs normal data well
6. **Attack data** has high reconstruction error → anomaly detected!
7. **Threshold** separates normal (low error) from anomalies (high error)

The magic: By learning to reconstruct normal patterns, the model automatically identifies anything that doesn't fit those patterns as an anomaly!

5. Model Architecture and Training Configuration {#model-details}

Before training, let's understand the complete model architecture, data splits, and training configuration.

5.1 Data Split Strategy

The dataset is split as follows:

- **Training Set:** 80% of NORMAL traffic only (used to train the autoencoder)
- **Validation Set:** 20% of NORMAL traffic (used for monitoring training and early stopping)
- **Test Set:** 20% of NORMAL traffic + ALL ATTACK traffic (used for final evaluation)

Why train on normal data only?

- Autoencoders learn to reconstruct what they've seen during training
- By training only on normal traffic, the model learns the "normal" pattern
- When attack traffic is presented, it cannot be reconstructed well → high reconstruction error → anomaly detected

5.2 Model Architecture

The autoencoder consists of two main parts:

Encoder (Compression):

```
Input (47 features)
→ Dense(64) + ReLU + Dropout(0.2)
→ Dense(32) + ReLU + Dropout(0.2)
→ Encoded Representation (32 dimensions)
```

Decoder (Reconstruction):

```
Encoded Representation (32 dimensions)
→ Dense(32) + ReLU + Dropout(0.2)
→ Dense(64) + ReLU + Dropout(0.2)
→ Output (47 features) + Sigmoid
```

Key Design Choices:

- **Encoding Dimension (32):** Compresses 47 features to 32, forcing the model to learn essential patterns
- **Hidden Layers (64, 32):** Gradual compression/expansion for better feature learning
- **Dropout (0.2):** 20% of neurons randomly disabled during training to prevent overfitting
- **Sigmoid Activation:** Output layer uses sigmoid to ensure reconstructed values are in [0,1] range (matching scaled input)

5.3 Loss Function

Mean Squared Error (MSE) Loss:

The model uses MSE to measure reconstruction error:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Where:

- x_i = original input feature value
- $\hat{x}_i$ = reconstructed feature value
- n = number of features (47)

For a batch of samples:
$$\text{MSE}_{\text{batch}} = \frac{1}{m \times n} \sum_{j=1}^m \sum_{i=1}^n (x_{ij} - \hat{x}_{ij})^2$$

Where:

- m = batch size
- n = number of features

Why MSE?

- Penalizes large reconstruction errors more than small ones
- Works well with normalized/scaled data
- Provides a clear measure of how well the model reconstructs input

Additional Metric: Mean Absolute Error (MAE)
$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |x_i - \hat{x}_i|$$

MAE is tracked during training but not used for optimization (only MSE is).

5.4 Anomaly Score Calculation

After training, the reconstruction error serves as the anomaly score:

$$\text{Anomaly Score} = \text{MSE}(x, \hat{x}) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2$$

Threshold Selection:

- Threshold = 95th percentile of reconstruction errors on normal test data
- Any sample with score > threshold → classified as ANOMALY
- Any sample with score ≤ threshold → classified as NORMAL

Why 95th percentile?

- Captures 95% of normal traffic as normal
- Allows 5% false positive rate (some normal traffic flagged as anomaly)
- Balances detection rate vs false alarms

5.5 Training Configuration

- **Optimizer:** Adam (Adaptive Moment Estimation)
 - Learning rate: Default (0.001)
 - Adaptive learning rate that adjusts per parameter
- **Batch Size:** 128 samples per batch
 - Larger batches provide more stable gradients
- **Epochs:** 50 maximum epochs
 - Training stops early if validation loss doesn't improve
- **Validation:** 20% of training data
 - Used to monitor overfitting
 - Early stopping based on validation loss
- **Shuffle:** Enabled
 - Randomizes sample order each epoch for better learning

5.6 Training Process

1. **Forward Pass:** Input → Encoder → Decoder → Reconstructed Output
2. **Loss Calculation:** MSE between input and reconstructed output
3. **Backward Pass:** Compute gradients using backpropagation
4. **Optimization:** Update weights using Adam optimizer

5. **Validation:** Evaluate on validation set to monitor generalization
6. **Early Stopping:** Stop if validation loss doesn't improve for several epochs

6. Train Autoencoder {#training}

Now let's train the autoencoder model on normal traffic data.

```
# Create and compile autoencoder
input_dim = X_train.shape[1]
model = Autoencoder(input_dim=input_dim, encoding_dim=32, hidden_dims=[64, 32])
model.compile_model()

# Display model architecture
model.autoencoder.summary()

# Train the model
print(f"\n{' '*60}")
print("TRAINING AUTOENCODER")
print(f"{' '*60}")

start_time = time.time()
history = model.train(X_train, X_val, epochs=EPOCHS, batch_size=BATCH_SIZE, verbose=1)
training_time = time.time() - start_time

print(f"\n✓ Training completed in {training_time:.2f} seconds")
```


✓ Autoencoder compiled successfully!
Model: "functional_6"

Layer (type)	Output Shape	Param #
input_layer_3 (InputLayer)	(None, 75)	0
dense_15 (Dense)	(None, 64)	4,864
dropout_12 (Dropout)	(None, 64)	0
dense_16 (Dense)	(None, 32)	2,080
dropout_13 (Dropout)	(None, 32)	0
encoded (Dense)	(None, 32)	1,056
dense_17 (Dense)	(None, 32)	1,056
dropout_14 (Dropout)	(None, 32)	0
dense_18 (Dense)	(None, 64)	2,112
dropout_15 (Dropout)	(None, 64)	0
dense_19 (Dense)	(None, 75)	4,875

Total params: 16,043 (62.67 KB)
Trainable params: 16,043 (62.67 KB)
Non-trainable params: 0 (0.00 B)

```
=====
TRAINING AUTOENCODER
=====
Epoch 1/50
219/219 ----- 4s 8ms/step - loss: 0.5712 - mae: 0.4830 - val_loss: 0.3621 - val_mae: 0.2786
Epoch 2/50
219/219 ----- 1s 7ms/step - loss: 0.3733 - mae: 0.2839 - val_loss: 0.3309 - val_mae: 0.2560
Epoch 3/50
219/219 ----- 2s 5ms/step - loss: 0.3492 - mae: 0.2652 - val_loss: 0.3265 - val_mae: 0.2519
Epoch 4/50
219/219 ----- 1s 4ms/step - loss: 0.3443 - mae: 0.2605 - val_loss: 0.3238 - val_mae: 0.2478
Epoch 5/50
219/219 ----- 1s 4ms/step - loss: 0.3404 - mae: 0.2563 - val_loss: 0.3224 - val_mae: 0.2458
Epoch 6/50
219/219 ----- 1s 4ms/step - loss: 0.3390 - mae: 0.2543 - val_loss: 0.3211 - val_mae: 0.2440
Epoch 7/50
219/219 ----- 1s 4ms/step - loss: 0.3366 - mae: 0.2517 - val_loss: 0.3200 - val_mae: 0.2422
Epoch 8/50
219/219 ----- 1s 4ms/step - loss: 0.3353 - mae: 0.2501 - val_loss: 0.3185 - val_mae: 0.2405
Epoch 9/50
219/219 ----- 1s 4ms/step - loss: 0.3343 - mae: 0.2488 - val_loss: 0.3177 - val_mae: 0.2398
Epoch 10/50
219/219 ----- 1s 4ms/step - loss: 0.3330 - mae: 0.2477 - val_loss: 0.3175 - val_mae: 0.2390
Epoch 11/50
219/219 ----- 1s 4ms/step - loss: 0.3324 - mae: 0.2467 - val_loss: 0.3168 - val_mae: 0.2384
Epoch 12/50
219/219 ----- 2s 7ms/step - loss: 0.3315 - mae: 0.2459 - val_loss: 0.3164 - val_mae: 0.2374
Epoch 13/50
219/219 ----- 2s 5ms/step - loss: 0.3309 - mae: 0.2452 - val_loss: 0.3163 - val_mae: 0.2372
Epoch 14/50
```

✓ Training History Visualization

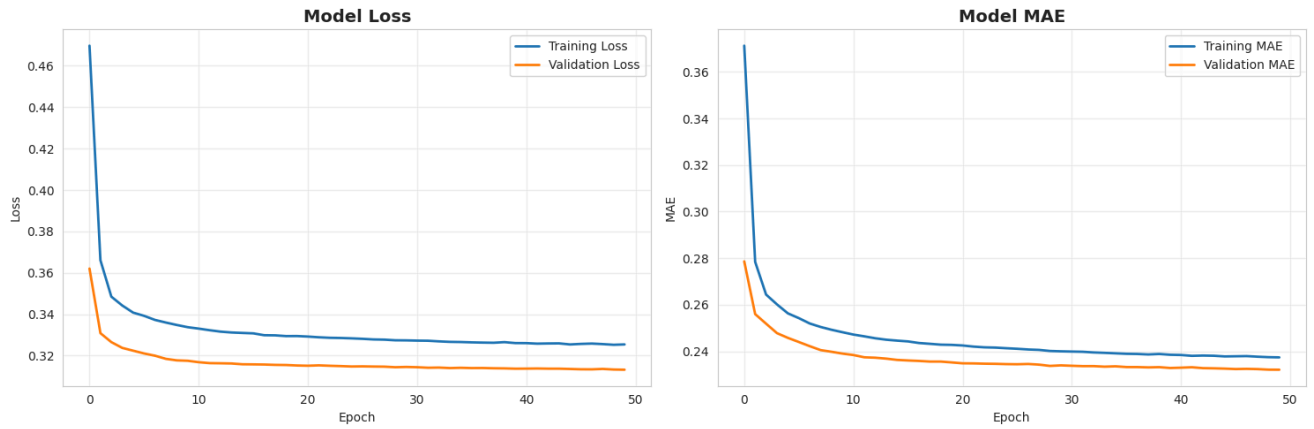
```
# Plot training history
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(15, 5))

ax1.plot(history.history['loss'], label='Training Loss', linewidth=2)
ax1.plot(history.history['val_loss'], label='Validation Loss', linewidth=2)
ax1.set_title('Model Loss', fontsize=14, fontweight='bold')
ax1.set_xlabel('Epoch')
ax1.set_ylabel('Loss')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.plot(history.history['mae'], label='Training MAE', linewidth=2)
ax2.plot(history.history['val_mae'], label='Validation MAE', linewidth=2)
ax2.set_title('Model MAE', fontsize=14, fontweight='bold')
ax2.set_xlabel('Epoch')
ax2.set_ylabel('MAE')
ax2.legend()
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

-----
Epoch 27/50
```



```
Epoch 37/50
219/219 ————— 2s 4ms/step - loss: 0.3255 - mae: 0.2383 - val_loss: 0.3140 - val_mae: 0.2332
Epoch 38/50
219/219 ————— 1s 4ms/step - loss: 0.3255 - mae: 0.2382 - val_loss: 0.3139 - val_mae: 0.2331
Epoch 39/50
219/219 ————— 1s 4ms/step - loss: 0.3259 - mae: 0.2383 - val_loss: 0.3139 - val_mae: 0.2332
Epoch 40/50
219/219 ————— 1s 4ms/step - loss: 0.3253 - mae: 0.2381 - val_loss: 0.3137 - val_mae: 0.2328
Epoch 41/50
219/219 ————— 1s 4ms/step - loss: 0.3253 - mae: 0.2379 - val_loss: 0.3137 - val_mae: 0.2330
Epoch 42/50
219/219 ————— 1s 4ms/step - loss: 0.3249 - mae: 0.2376 - val_loss: 0.3138 - val_mae: 0.2331
Epoch 43/50
219/219 ————— 1s 4ms/step - loss: 0.3252 - mae: 0.2377 - val_loss: 0.3137 - val_mae: 0.2328
Epoch 44/50
219/219 ————— 1s 4ms/step - loss: 0.3254 - mae: 0.2377 - val_loss: 0.3137 - val_mae: 0.2327
Epoch 45/50
```

7. Evaluation {#evaluation}

Set threshold and evaluate on test set (normal + attack traffic)

```
# Set threshold using normal data from test set
X_normal_test = X_test[y_test == 0]
threshold = model.set_threshold(X_normal_test, percentile=95)

# Evaluate model
results = model.evaluate(X_test, y_test)

# Store scores for visualization
normal_scores = results['scores'][y_test == 0]
anomaly_scores = results['scores'][y_test == 1]
```

✓ Training completed in 00.40 seconds
Anomaly threshold set to: 1.471956

CLASSIFICATION REPORT

	precision	recall	f1-score	support
Normal	1.00	0.95	0.97	8744
Anomaly	0.93	1.00	0.97	6280
accuracy			0.97	15024
macro avg	0.97	0.97	0.97	15024
weighted avg	0.97	0.97	0.97	15024

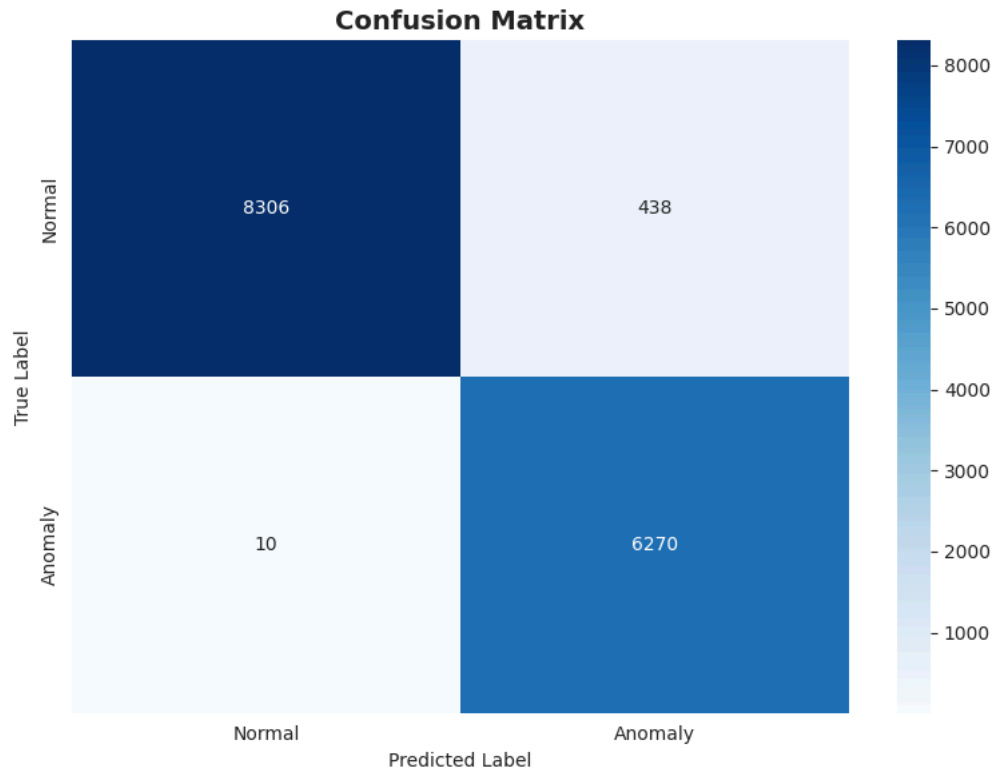
Confusion Matrix:
[[8306 438]
[10 6270]]

ROC AUC: 0.9868
Accuracy: 0.9702
Precision: 0.9347
Recall: 0.9984
F1-Score: 0.9655

Visualization: Confusion Matrix, ROC Curve, and Anomaly Scores

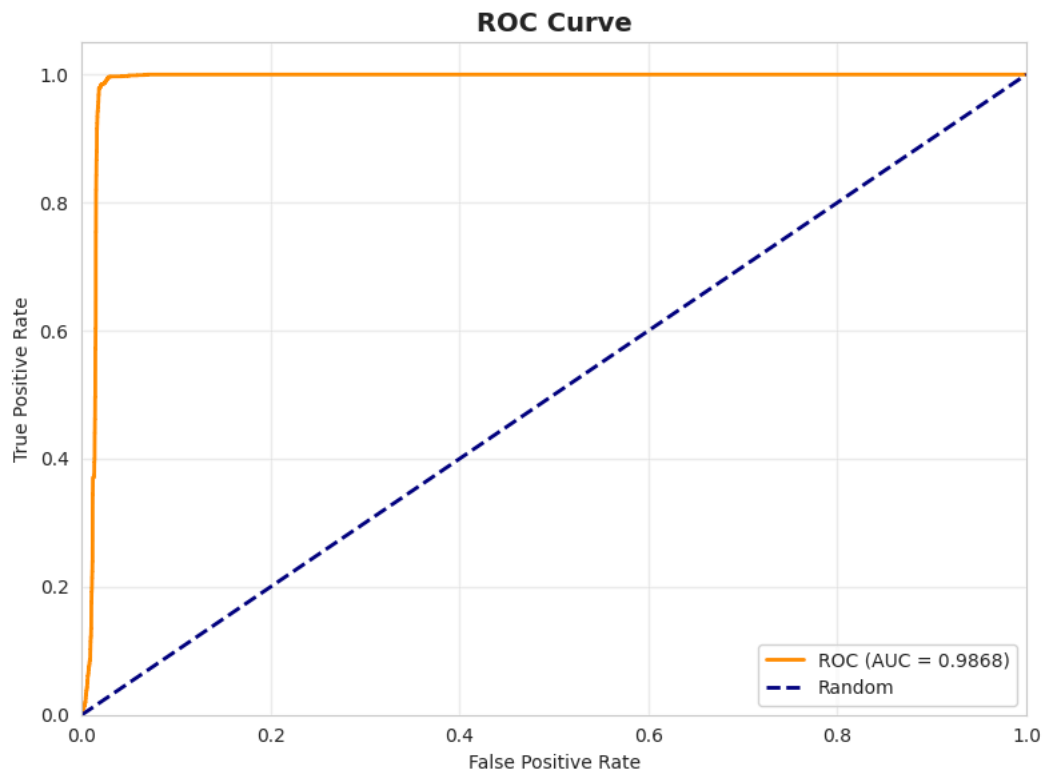
Visualization 1: Confusion Matrix

```
# 1. Confusion Matrix
plt.figure(figsize=(8, 6))
cm = results['confusion_matrix']
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=True,
            xticklabels=['Normal', 'Anomaly'], yticklabels=['Normal', 'Anomaly'])
plt.title('Confusion Matrix', fontsize=14, fontweight='bold')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()
```



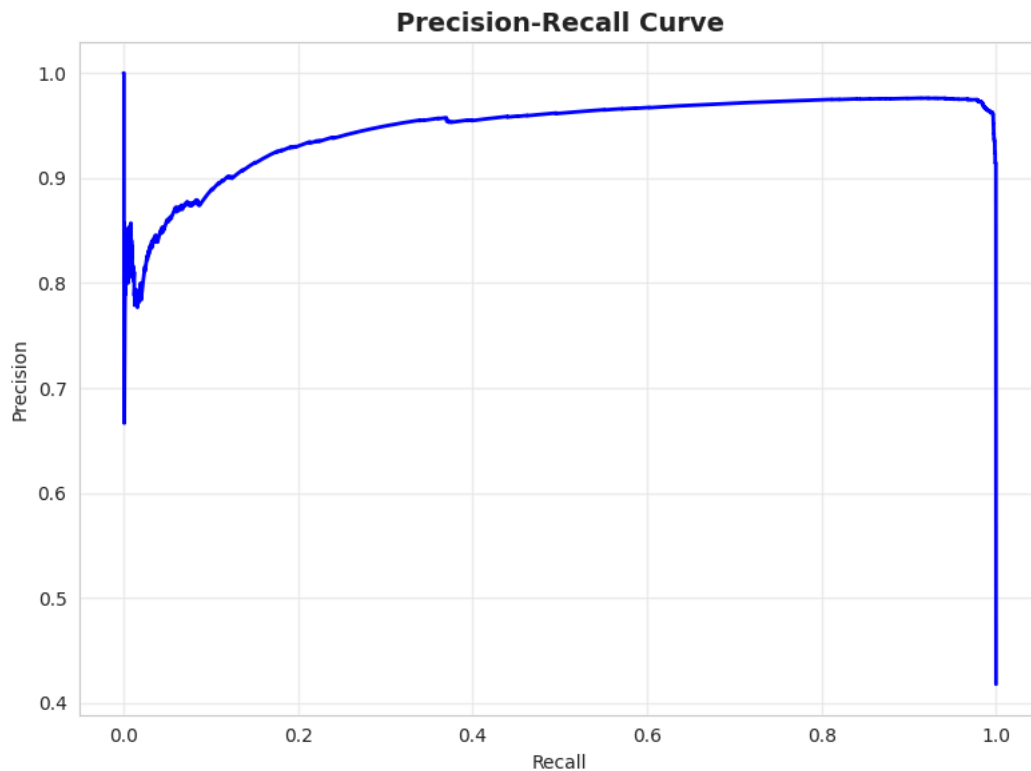
Visualization 2: ROC Curve

```
# 2. ROC Curve
plt.figure(figsize=(8, 6))
fpr, tpr, _ = roc_curve(y_test, results['scores'])
roc_auc_val = auc(fpr, tpr)
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC (AUC = {roc_auc_val:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve', fontsize=14, fontweight='bold')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



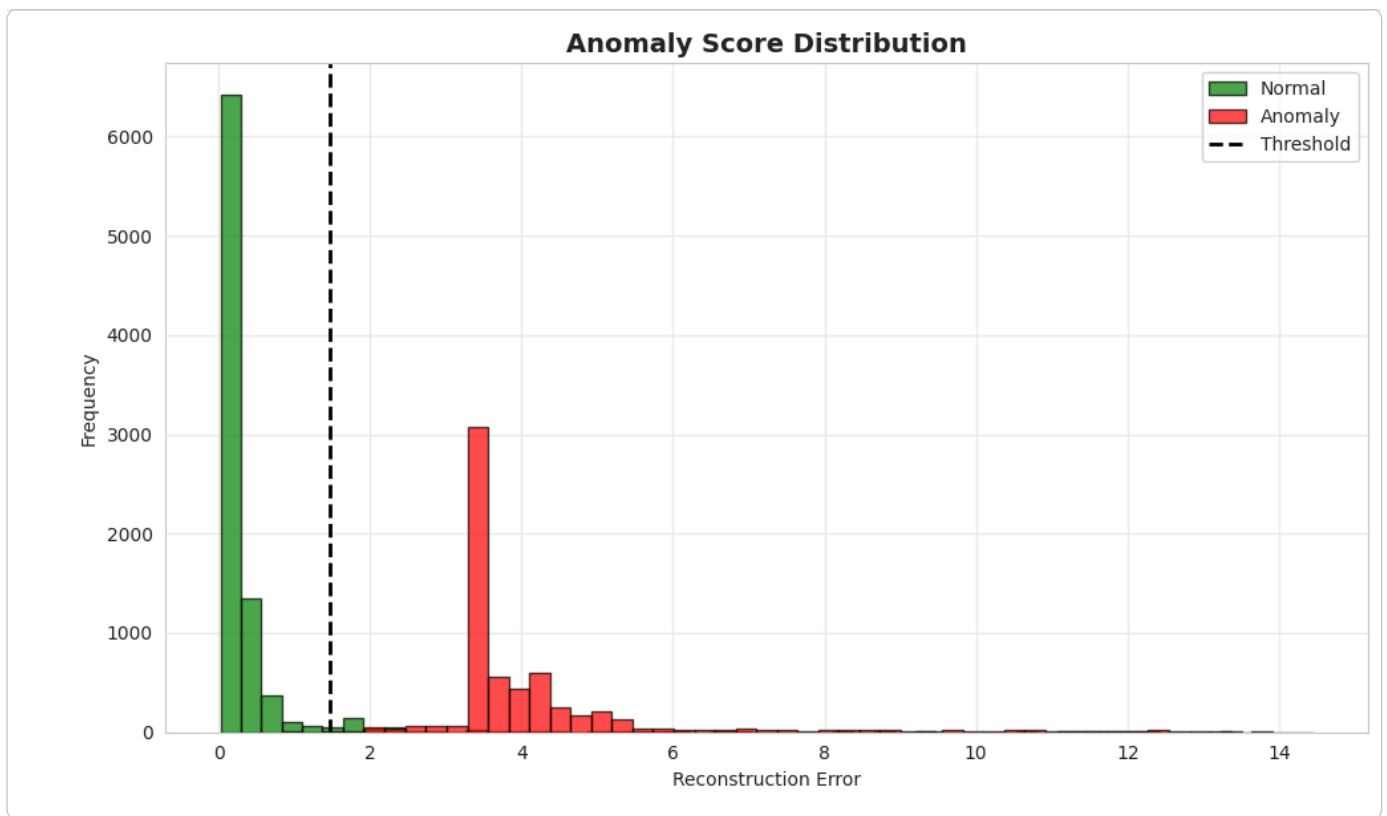
Visualization 3: Precision-Recall Curve

```
# 3. Precision-Recall Curve
plt.figure(figsize=(8, 6))
precision, recall, _ = precision_recall_curve(y_test, results['scores'])
plt.plot(recall, precision, color='blue', lw=2)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



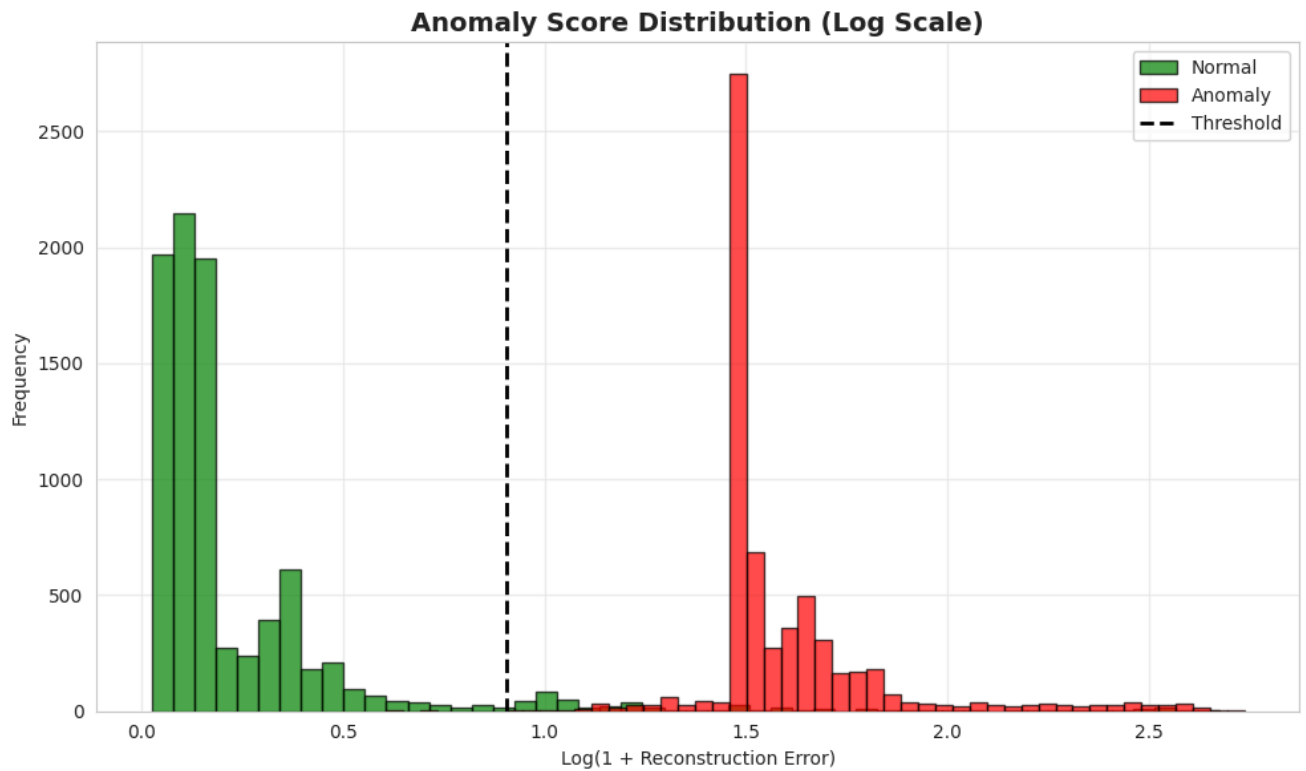
Visualization 4: Anomaly Score Distribution

```
# 4. Anomaly Score Distribution
plt.figure(figsize=(10, 6))
plt.hist(normal_scores, bins=50, alpha=0.7, label='Normal', color='green', edgecolor='black')
plt.hist(anomaly_scores, bins=50, alpha=0.7, label='Anomaly', color='red', edgecolor='black')
plt.axvline(threshold, color='black', linestyle='--', linewidth=2, label=f'Threshold')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.title('Anomaly Score Distribution', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



Visualization 5: Anomaly Score Distribution (Log Scale)

```
# 5. Anomaly Score Distribution (Log Scale)
plt.figure(figsize=(10, 6))
plt.hist(np.log1p(normal_scores), bins=50, alpha=0.7, label='Normal', color='green', edgecolor='black')
plt.hist(np.log1p(anomaly_scores), bins=50, alpha=0.7, label='Anomaly', color='red', edgecolor='black')
plt.axvline(np.log1p(threshold), color='black', linestyle='--', linewidth=2, label='Threshold')
plt.xlabel('Log(1 + Reconstruction Error)')
plt.ylabel('Frequency')
plt.title('Anomaly Score Distribution (Log Scale)', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

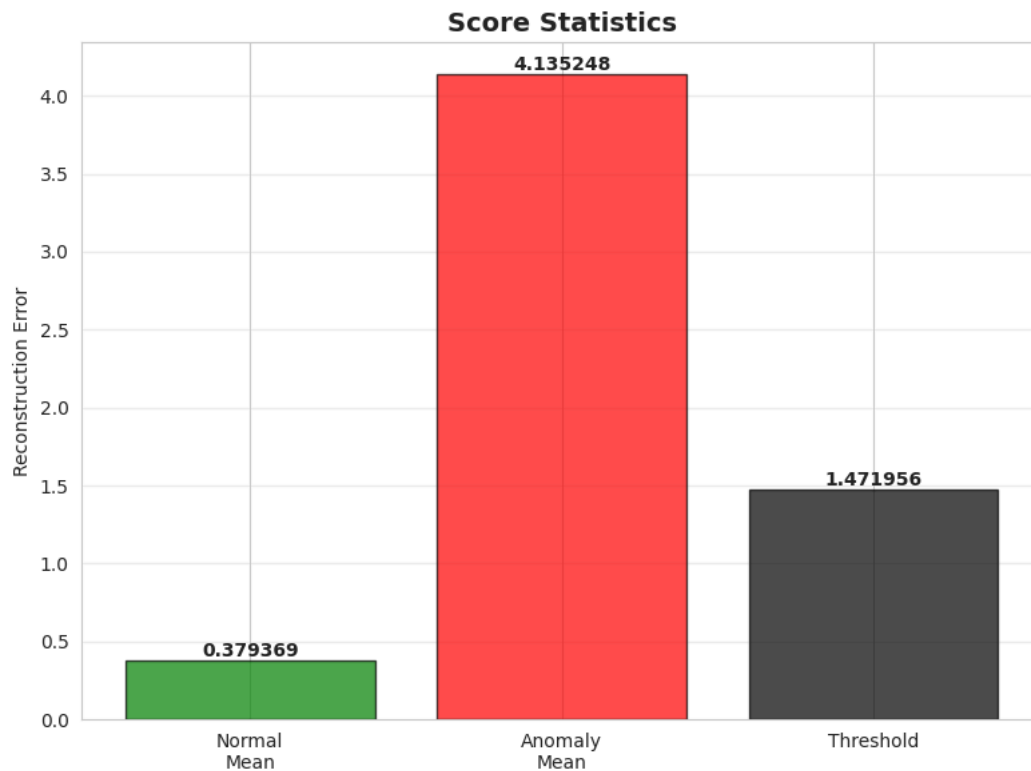


Visualization 6: Score Statistics

```
# 6. Score Statistics
plt.figure(figsize=(8, 6))
stats_data = [normal_scores.mean(), anomaly_scores.mean(), threshold]
stats_labels = ['Normal\nMean', 'Anomaly\nMean', 'Threshold']
colors_stats = ['green', 'red', 'black']
bars = plt.bar(stats_labels, stats_data, color=colors_stats, alpha=0.7, edgecolor='black')
plt.ylabel('Reconstruction Error')
plt.title('Score Statistics', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3, axis='y')

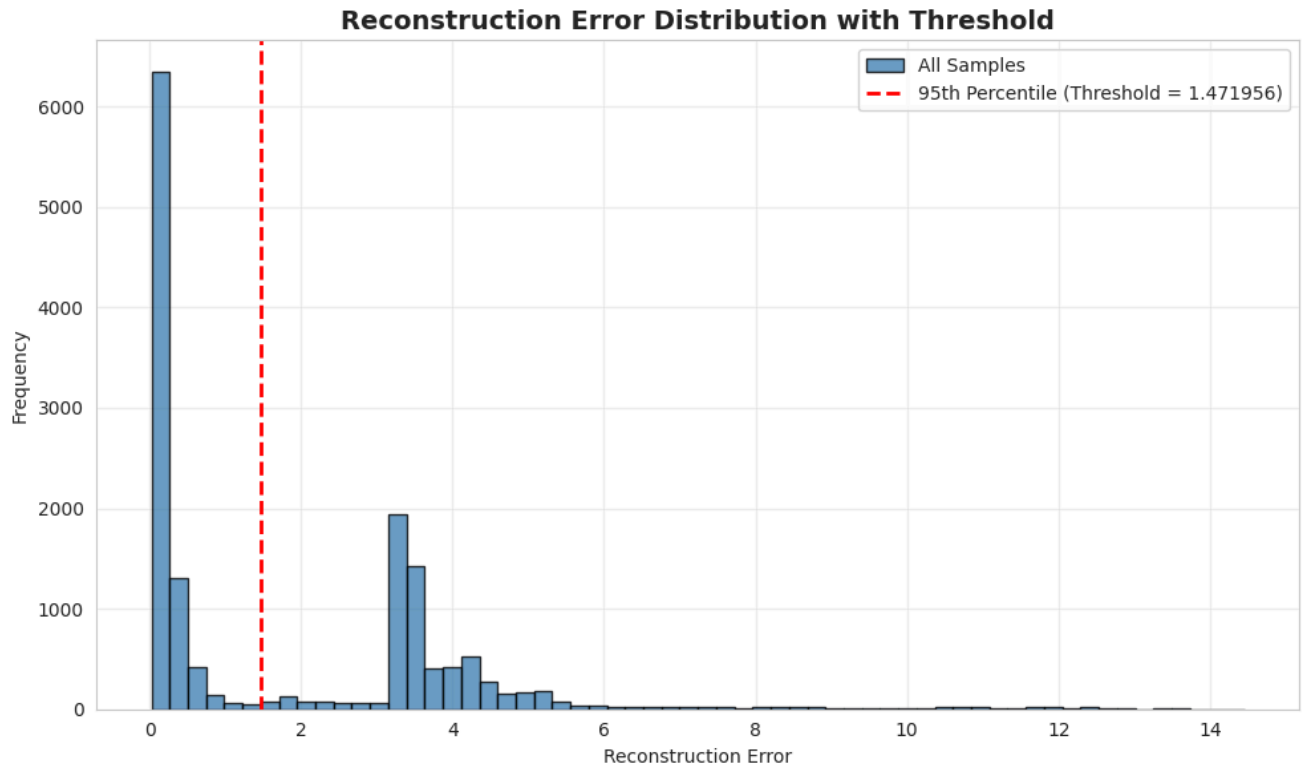
# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.6f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()
```



✓ Visualization: Reconstruction Error Distribution with Threshold

```
# Reconstruction error distribution with threshold
plt.figure(figsize=(10, 6))
all_scores = results['scores']
plt.hist(all_scores, bins=60, alpha=0.8, color='steelblue', edgecolor='black', label='All Samples')
plt.axvline(threshold, color='red', linestyle='--', linewidth=2, label=f'95th Percentile (Threshold = {threshold:.6f}')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.title('Reconstruction Error Distribution with Threshold', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



```
fig = plt.figure(figsize=(18, 12))

# 1. Confusion Matrix
ax1 = plt.subplot(2, 3, 1)
cm = results['confusion_matrix']
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', cbar=True,
            xticklabels=['Normal', 'Anomaly'], yticklabels=['Normal', 'Anomaly'])
plt.title('Confusion Matrix', fontsize=14, fontweight='bold')
plt.ylabel('True Label')
plt.xlabel('Predicted Label')

# 2. ROC Curve
ax2 = plt.subplot(2, 3, 2)
fpr, tpr, _ = roc_curve(y_test, results['scores'])
roc_auc_val = auc(fpr, tpr)
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC (AUC = {roc_auc_val:.4f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve', fontsize=14, fontweight='bold')
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)

# 3. Precision-Recall Curve
ax3 = plt.subplot(2, 3, 3)
precision, recall, _ = precision_recall_curve(y_test, results['scores'])
plt.plot(recall, precision, color='blue', lw=2)
plt.xlabel('Recall')
plt.ylabel('Precision')
plt.title('Precision-Recall Curve', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3)

# 4. Anomaly Score Distribution
ax4 = plt.subplot(2, 3, 4)
plt.hist(normal_scores, bins=50, alpha=0.7, label='Normal', color='green', edgecolor='black')
plt.hist(anomaly_scores, bins=50, alpha=0.7, label='Anomaly', color='red', edgecolor='black')
plt.axvline(threshold, color='black', linestyle='--', linewidth=2, label=f'Threshold')
plt.xlabel('Reconstruction Error')
plt.ylabel('Frequency')
plt.title('Anomaly Score Distribution', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)
```

```

# 5. Anomaly Score Distribution (Log Scale)
ax5 = plt.subplot(2, 3, 5)
plt.hist(np.log1p(normal_scores), bins=50, alpha=0.7, label='Normal', color='green', edgecolor='black')
plt.hist(np.log1p(anomaly_scores), bins=50, alpha=0.7, label='Anomaly', color='red', edgecolor='black')
plt.axvline(np.log1p(threshold), color='black', linestyle='--', linewidth=2, label='Threshold')
plt.xlabel('Log(1 + Reconstruction Error)')
plt.ylabel('Frequency')
plt.title('Anomaly Score Distribution (Log Scale)', fontsize=14, fontweight='bold')
plt.legend()
plt.grid(True, alpha=0.3)

# 6. Score Statistics
ax6 = plt.subplot(2, 3, 6)
stats_data = [normal_scores.mean(), anomaly_scores.mean(), threshold]
stats_labels = ['Normal\nMean', 'Anomaly\nMean', 'Threshold']
colors_stats = ['green', 'red', 'black']
bars = plt.bar(stats_labels, stats_data, color=colors_stats, alpha=0.7, edgecolor='black')
plt.ylabel('Reconstruction Error')
plt.title('Score Statistics', fontsize=14, fontweight='bold')
plt.grid(True, alpha=0.3, axis='y')

# Add value labels on bars
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2., height,
             f'{height:.6f}', ha='center', va='bottom', fontweight='bold')

plt.tight_layout()
plt.show()

# Print summary statistics
print(f"\n{'='*60}")
print("SCORE STATISTICS")
print(f"{'='*60}")
print(f"Normal scores - Mean: {normal_scores.mean():.6f}, Std: {normal_scores.std():.6f}")
print(f"Anomaly scores - Mean: {anomaly_scores.mean():.6f}, Std: {anomaly_scores.std():.6f}")
print(f"Threshold: {threshold:.6f} (95th percentile of normal scores)")

# Explanation of threshold position
print(f"\n{'='*60}")
print("THRESHOLD ANALYSIS")
print(f"{'='*60}")
print(f"Threshold position relative to means:")
print(f" - Threshold > Normal Mean: ✓ Expected (threshold is 95th percentile)")
if threshold > anomaly_scores.mean():
    print(f" - Threshold > Anomaly Mean: ⚠ This can happen if:")
    print(f"     • Some attacks are easy to detect (very high scores)")
    print(f"     • Some attacks are harder to detect (lower scores)")
    print(f"     • The mean is pulled down by easier-to-detect attacks")
    print(f"     • The distribution has high variance")
    print(f" - Check the histogram: if there's good separation, the model is working!")
    print(f" - The threshold correctly separates most anomalies (see confusion matrix)")
else:
    print(f" - Threshold < Anomaly Mean: ✓ Good separation")
print(f"\nKey Insight: The threshold is positioned to minimize false positives")
print(f"while maximizing true positives. The visual separation in the histogram")
print(f"is more important than the mean comparison.")

```